

队伍编号	202504661
选题	A 题

康养城市建设与优化配置建模

摘要

康养城市建设不仅是国家层面的战略部署，也是地方发展的实际需求。本研究旨在解决如何在有限的资源条件下，优化康养设施布局，提升康养服务质量的问题，以推动智慧养老与康养产业的深度融合。

针对问题 1，本研究以北京市 16 个区为对象，围绕康养资源分布的现状分析与优化需求展开。通过收集 2023 年北京市各区康养资源及居民健康状况数据，进行数据处理与可视化分析，明确了各区康养资源和居民健康指标的空间分布特征。结果显示，中心城区康养资源富集，远郊区资源匮乏，且居民健康指标存在显著区域差异；进一步构建**康养资源指数 (R_i)**和**居民健康指数 (H_i)**模型，采用熵值法确定权重，整合多维度指标。经相关性分析发现，剔除东城区、西城区后，其余 14 区康养资源指数与居民健康指数呈正相关，相关系数 0.6834，但**整体方差达 0.7075**，表明**资源分配不均衡**。引入资源匹配度 (PR_i) 评估发现，超半数区域存在**人均资源不足**问题；为优化资源布局，构建以最小化康养资源指数方差和平均可达时间为目标的优化模型，采用**SLSQP 算法**求解。结果表明，优化后需减少东城区、西城区、石景山区的康养资源量，增加朝阳区、丰台区等区域的资源配置，以提升服务公平性与可达性，为康养城市建设提供科学规划依据。

针对问题 2，本研究构建了一套完整的评价体系与优化方法，以评估和提升北京市康养城市发展水平。具体而言，首先从多个维度选取代表性定量指标，运用**熵权法与 TOPSIS 方法**构建多维综合评价模型，对北京市各区的康养城市发展水平进行量化评估。结果表明，不同区发展水平差异较大，存在明显的两极分化。在此基础上，创新性地引入强化学习方法，定量计算并提出各区在各维度上的具体发展调整建议，形成一套可量化、可操作的发展优化方案。

针对问题 3，本研究构建康养资源**多目标优化配置模型**，以北京市 16 区为对象，设定覆盖率、效率最大化与成本最小化目标函数，结合服务半径、预算等约束条件，采用**粒子群优化 (PSO)**、**差分进化算法 (DE)**、**NSGAIII 算法**求解。结果表明，海淀区、房山区等区域需针对性增加医疗、养老等设施数量（如房山区医疗设施需增 3111 家），部分核心城区设施配置可维持合理水平。研究提出区域差异化政策、多元资金投入及智慧康养技术等优化策略，为康养资源高效配置提供科学方案。

关键词：康养城市建设；北京市；康养资源；居民健康指数；SLSQP 法；TOPSIS 法；多目标优化

1. 问题重述

1.1 问题背景

随着全球老龄化进程加速，康养城市建设已成为应对人口结构变化、提升居民生活质量的重要战略方向。2025 年全国两会期间，多位代表委员围绕康养产业转型、智慧养老及医养融合等议题提出关键建议，如盘活“空置房”用于康养产业、发展数字家庭技术等。安徽省人民政府发布《关于支持康养产业高质量发展的意见》，强调康养产业与医疗、旅游、文化的深度融合；地方实践中，广德市推进“康养名城”战略，通过优化设施布局、提升智慧养老水平等打造康养目的地，吉林省亦将旅居康养纳入地方文旅产业发展规划。此类政策与实践表明，康养城市建设既是国家战略部署，亦是地方经济社会发展的现实需求。

然而，当前康养城市建设面临多重挑战。康养资源（如医疗设施、养老机构、文化设施等）在区域间分布不均衡，智慧养老技术应用滞后，医养融合深度不足。在资源总量有限的约束下，如何通过科学规划优化康养设施布局、提升服务可达性与公平性，成为亟待解决的关键问题。

1.2 问题提出

针对康养城市建设问题，要求我们通过参考并收集相关数据，比如：2024 年中国康养城市 100 强名单，国家统计局数据（老年人口数据、居民收入与消费支出、生态环境、医疗水平）、人力资源和社会保障部（养老保险基金支出）以及一些国内外公开的数据，通过数学建模（或算法）方法，解决以下问题：

（1）问题一是康养资源分布的现状分析与优化需求问题。要求我们收集或使用某城市不同区域模拟数据，包括康养资源数据（医疗设施、养老机构、公园绿地、文化设施的数量与分布密度）以及居民健康状况数据（平均寿命、慢性病发病率、健康满意度等），进而开展研究。评估所选城市的康养资源分布的空间特征，判断康养资源的富集区和匮乏区，通过分析康养资源分布与居民健康指标的相关性，判断当前康养资源分布的合理性。在此基础上，针对康养资源分布不均衡的问题，提出优化布局的初步思路，重点提升康养服务的公平性与可达性。

（2）问题二是康养城市综合评价模型的构建与应用问题。康养城市建设已成为提升城市竞争力与居民生活质量的重要方向，需要科学的评价体系来衡量其发展水平。现需构建一个适用于康养城市的综合评价模型，具体表现为，首先，需从康养资源丰富度（如医疗设施数量、养老机构床位密度等）、居民健康状况（包括平均寿命、慢性病发病率等）、环境质量（如空气质量指数、绿地覆盖率等）、经济发展水平（人均 GDP、康养产业占比等）等多个核心维度，合理选取并定义具体的评价指标；其次，利用该模型对目标城市的康养水平进行量化计算，通过数据处理与分析，明确其在各维度上的优势与不足；最后，基于分析结果，提出具有针对性的改进建议，包括资源优化配置方案、环境治理措施、产业发展策略等，为康养城市的科学建设提供可操作的决策依据

（3）问题三是康养资源优化配置模型的建立与实施策略问题。康养城市建设的核心在于高效配置康养资源，以提升康养服务的覆盖率和资源利用效率，同时在有限的资源条件下最小化成本。当前康养城市建设面临诸多挑战，如康养资源分布不均衡、智慧养老技术应用不足、医养融合深度不够等。这些问题导致部分区域康养资源过剩，而另一些区域则资源匮乏，影响了康养服务的公平性和可达性。因此，需要建立一个科学的康养资源优化配置模型，通过多目标优化方法，合理规划康养设施的选址、数量和服务范围，以实现资源的最优配置。具体问题为如何在有限的资源条件下，最大化康养服务的覆盖率和资源利用效率，同时最小化成本。

2. 问题分析

问题一：针对问题 1 的要求，其求解可分为以下四步，首先选择某一典型城市为研究对象，收集并整理题目所需要的相关数据，并利用 Python 编程语言和 Geopandas 库进行数据处理和可视化分析，明确该城市各区康养资源和居民健康状况各指标的空间分布特征并进行初步的定性分析，判断康养资源分布的合理性；随后进行定量分析，综合各个独立指标，建立康养资源指数和居民健康指数模型，用以反映康养资源服务压力和居民健康水平；之后，构建康养资源合理性定量评价模型，使用采用 Pearson 相关性分析法，明确康养资源指数与居民健康指数的关系，分析资源配置不合理的情况，为了进一步探究资源分配的均衡性，基于康养资源指数，探究其离散程度；最后构建资源分配优化模型，分析该城市各区资源匹配度，确定目标函数和约束条件，并确定合适的优化方法进行计算，确定资源分配方案，以解决资源分布不均衡的问题，提高康养服务的公平性和可达性。

问题二：针对问题 2 的要求，核心任务是建立一个合理、全面的综合评价模型，量化衡量城市康养发展水平，分析当前城市康养水平的优势与不足，并为城市康养规划提供决策支持。对于康养城市发展水平的评价，需从多维度构建综合评价体系。一方面，要考量康养资源的丰富度与配置效率，涵盖医疗设施的可达性、养老机构的服务能力、公园绿地的空间分布以及文化设施的覆盖密度等，确保资源在城乡、区域间均衡布局；另一方面，需聚焦居民健康状况，包括平均寿命、慢性病发病率等。同时，环境质量是关键基础，涉及空气质量、绿地面积等，直接影响居民的生活舒适度与健康风险。经济发展水平则为康养服务提供支撑，涉及人均 GDP、地区生产总值等，确保产业可持续发展。从而形成涵盖资源、健康、环境、经济等多维度的立体评价框架。因此，针对问题二，选取各个维度下具有代表性的定量指标，我们设计了基于熵权和 TOPSIS 方法的多维综合评价模型来评价北京市康养城市的发展水平，根据评价结果进行聚类对多个区进行发展水平等级划分，并通过多种可视化分析北京市不同地区发展水平及分布差异，最后创新性的应用了强化学习方法进行提出改进意见，该方法定量的指出了每个区未来应该做出的发展调整从而提高发展水平。

问题三：针对问题 3 的要求，本质为多目标优化问题。首先，应平衡研究城市各区最大化服务覆盖率、资源利用效率和最小化成本这三个目标，考虑设施选址等空间约束。构建模型时，确定决策变量，设好最大化覆盖率、效率和最小化成本的目标函数，服务半径、预算等约束条件。在多目标优化方法选择上，拟采用粒子群优化（PSO）、差分进化算法（DE）、第三代非支配排序遗传算法（NSGAIII）三种前沿优化算法对结果进行分析总结。最后依据三种算法模型结果与问题一结果进行对比分析、合理的提出具体的实施策略，包括政策建议、资金投入方向、技术创新措施等。

3. 模型假设与符号说明

3.1 模型基本假设

为了简化模型并使其更贴近实际情况，本研究做出以下假设：

（1）资源总量固定。假设北京市康养资源总量在优化过程中保持不变，即资源的增加或减少仅在不同区域之间进行调配；

（2）区域独立性。假设各区域之间的康养资源分配相互独立，不考虑跨区域的资源共享或协同效应。

（3）数据准确性。假设收集的康养资源和居民健康状况数据是准确且可靠的，能够真实反映各区域的实际情况。

（4）人口稳定性。假设在研究期间，各区域的人口数量保持相对稳定，不考虑大规模的人口流动对康养资源需求的影响。

3.2 符号说明

表 1 符号说明

符号	含义	单位
R_i	康养资源指数	-
M_i	医疗设施数量	家
E_i	养老机构数量	家
G_i	公园绿地数量	个
C_i	文化设施数量	家
P_i	区域内人口数量	万人
H_i	居民健康指数	-
L_i	人均平均寿命	岁
D_i	慢性病发病率	%
D_{ti}	慢性病过早死亡率	%
S_i	居民健康满意度	10 分制
σ_R^2	康养资源方差	-
PR_i	资源匹配度	-
C_j	第 j 设施建设成本	元
S_j	第 j 设施最大服务容量	-
d_{ij}	第 i 区域设施到 j 设施点距离	m
r_j	最大服务半径	-
B	投资预算	元
C_j	第 j 设施建设成本	元

4. 模型建立与求解

4.1 问题一模型建立与求解

4.1.1 数据与分布特征

（1）研究对象选择与数据收集

康养城市的建设离不开对现有康养资源的全面评估。北京作为中国的首都，人口规模庞大且老龄化程度高，截至 2023 年，60 岁及以上常住人口占比达 20.2%，65 岁及以上常住人口占比 14.9%，康养资源需求巨大。同时，北京市各区在经济发展水平、人口密度、土地利用等方面存在显著差异，这种差异导致各区对康养资源的需求和供给能力不同，研究其康养资源分布情况，能够更好地了解 and 解决区域间的康养资源不均衡问题，实现资源的优化配置。此外，北京市政府高度重视康养事业的发展，出台了一系列政策文件，为研究提供了明确的方向和依据。同时，北京的数据资源丰富且相对容易获取，能够为研究提供坚实的基础，确保研究结果的准确性和可靠性^[1,2]。

因此，本团队针对此问题，选择北京市各区作为康养资源研究对象，不仅具有重要的现实意义，还具有典型性和示范性，能够为其他城市提供借鉴和参考，推动全国康养事业的发展。本次收集并整理了 2023 年北京市东城区、西城区、朝阳区、丰台区、石景山区、海淀区、门头沟区、房山区、通州区、顺义区、昌平区、大兴区、怀柔区、平谷区、密云区及延庆区等 16 个区的康养资源分布情况（医疗设施、养老机构、公园绿地、文化设施等的数量和密度），以及居民健康状况数据（平均寿命，慢性病发病率，慢性病过早死亡率、健康满意度等）。部分数据如下表所示。

表 4.1 北京市 2023 年康养资源数据表

区	医疗设施数量 (家)	养老机构数量 (家)	公园绿地数量 (个)	文化设施数量 (家)
东城区	850	18	90	250
西城区	800	20	85	280
朝阳区	1200	120	160	300
丰台区	700	100	100	200
石景山区	400	50	70	100
海淀区	1100	110	150	280
门头沟区	180	30	40	50
房山区	500	60	60	100
通州区	500	70	90	120
顺义区	450	55	60	80
昌平区	600	65	120	110
大兴区	550	80	100	130
怀柔区	300	35	50	60
平谷区	250	30	48	50
密云区	200	40	45	70
延庆区	200	30	42	50

表 4.2 北京市 2023 年居民健康状况数据表

区	人均平均寿命 (岁)	慢性病发病率 (%)	慢性病过早死亡率 (%)	健康满意度 (10 分制)
东城区	83.2	28	9.8	9.2
西城区	83.1	27	9.9	9.1
朝阳区	82.9	26	10.1	9
丰台区	82	25	10.5	8.5
石景山区	82.1	25	10.4	8.6
海淀区	83	26	10	9.1
门头沟区	81.1	24	11.3	7.7
房山区	81	24	11.4	8
通州区	81.8	25	10.6	8.3
顺义区	81.9	25	10.5	8.4
昌平区	82.2	25	10.8	8.7
大兴区	81.7	25	10.7	8.2
怀柔区	81.5	24	10.9	8.1
平谷区	81.4	24	11	8
密云区	81.3	24	11.1	7.9
延庆区	81.2	24	11.2	7.8

(2) 数据来源与说明

①医疗设施数量选取社区医院数量数据。来源于《北京人必看：各城区医疗资源分析，6 成小区 500 米内有社区医院》，分布密度为数量除以面积计算所得。其他未直接获取到数据的区域，根据各区人口规模、经济发展水平、医疗资源整体分布趋势进行估算。假设医疗设施分布与人口分布大致成正比，依据各区常住人口数据，参考最新统计年鉴，结合已知城区医疗设施数量进行估算；

②养老机构数量。东城区数据来源于北京市东城区人民政府门户网站，其他区域根据《治理 | 空间视角下大都市区养老资源供需匹配分析及预测研究 —— 以北京为例》中各区养老机构分布情况描述，结合各区老年人口规模、养老产业发展基础估算。分布密度为数量除以面积计算所得；

③公园绿地数量。由于未获取到准确数据，根据各区生态定位、绿化建设投入和实际建成情况等因素进行估算。分布密度为数量除以面积计算所得；

④文化设施数量。东城区结合《北京市东城区人民政府门户网站》相关内容估算，其他区域根据各区文化底蕴、人口密度、文化事业发展规划等因素估算。分布密度为数量除以面积计算所得。

⑤人均平均寿命。基于北京市整体平均期望寿命 82.51 岁，结合各区医疗水平、生活环境差异微调。医疗资源丰富、生活环境好的区数值相对偏高；

⑥慢性病发病率、慢性病过早死亡率。参考北京市整体慢性病发病情况，结合各区居民生活习惯、工作压力、健康管理水平等因素估算。生活节奏快、压力大的区发病率相对较高；

⑦健康满意度。根据各区经济水平、医疗资源、环境质量、养老设施完善程度等综

合评估。经济发达、公共服务完善的区满意度相对较高。

(3) 数据可视化与定性分析

本研究使用了北京市各区的康养资源数据，包括医疗设施、养老机构、公园绿地和文化设施的数量。数据来源于北京市统计局和相关政府部门。地理空间数据采用北京市各区的行政区划图。利用 Python 编程语言和 Geopandas 库进行数据处理和可视化分析。

根据北京市康养资源分布图研究发现，医疗设施主要集中在中心城区，如东城区、西城区和朝阳区。这些区域人口密度高，对医疗服务需求较大，因此医疗设施数量较多。相比之下，远郊区如延庆区、怀柔区的医疗设施数量较少；养老机构主要集中在海淀区、朝阳区和丰台区。这些区域经济发展水平较高，养老服务需求旺盛。然而，远郊区的养老机构数量相对较少。公园绿地在各区分布较为均匀，但在海淀区、朝阳区和丰台区分布较为密集。这些区域的公园绿地数量较多，可能与城市规划和生态环境保护有关。文化设施主要集中在东城区、西城区和朝阳区。这些区域作为北京市的文化中心，拥有丰富的文化资源和设施。相比之下，远郊区的文化设施数量较少。

通过分析北京市各区康养资源的分布情况，发现资源分布存在显著的区域差异。中心城区的医疗设施、养老机构和文化设施数量较多，而远郊区的资源相对匮乏。公园绿地在各区分布较为均匀。这些发现为优化北京市康养资源配置提供了科学依据，建议政府在资源分配时考虑区域差异，促进资源均衡发展。

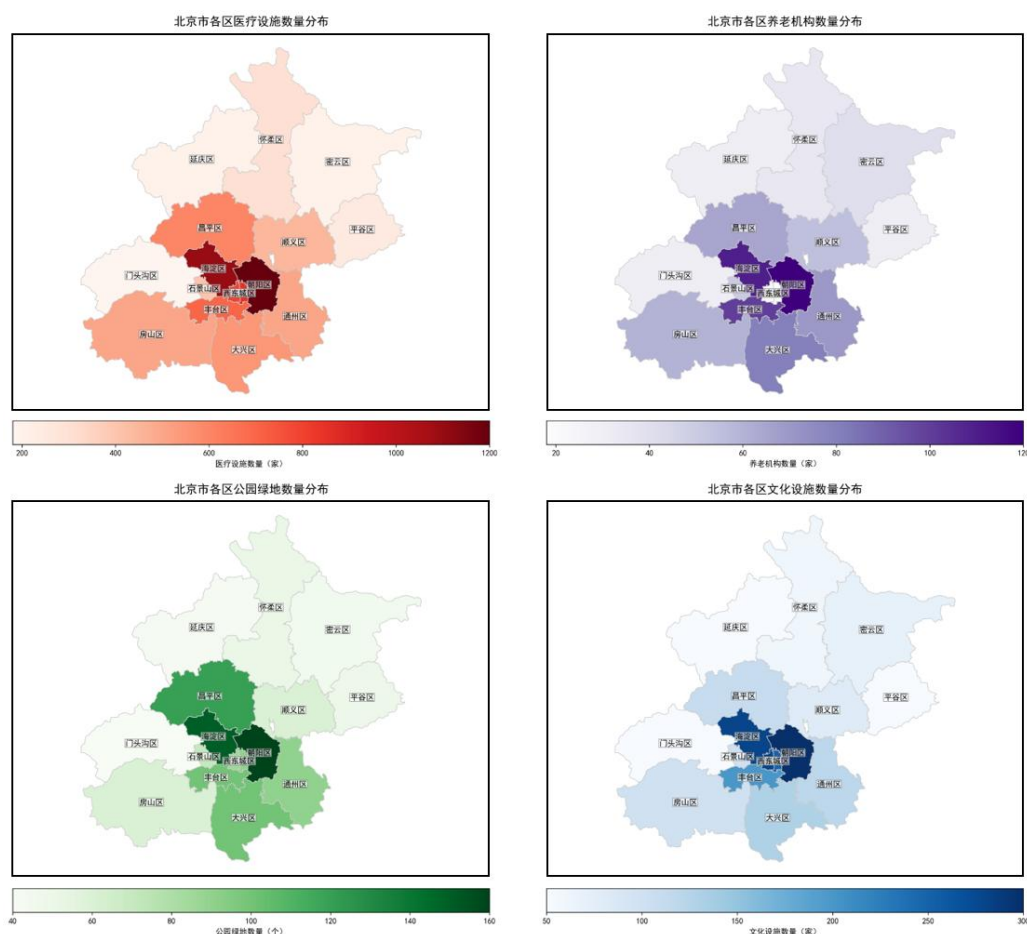


图 4.1 北京市各区康养资源数据分布图

根据北京市康养资源分布图研究发现，北京市各区人均平均寿命呈现明显的区域差异。中心城区部分区域（如西城区、东城区）人均平均寿命相对较高，可能得益于优质的医疗资源集中、公共卫生服务体系完善。而部分远郊区县人均平均寿命略低，或许与医疗资源相对匮乏、生活环境及健康管理意识等因素有关；慢性病发病率在北京市各区也存在差异。中心城区部分区域发病率相对较高，这可能与人口老龄化程度较高、生活节奏快、压力大以及不良生活方式流行有关；健康满意度方面，中心城区部分区域满意度较高，可能源于居民对医疗服务的可及性和质量认可度高，同时丰富的文化娱乐等配套设施也提升了居民的整体生活感受。而部分区域满意度较低，可能与环境质量、医疗服务的供需矛盾等因素有关；慢性病过早死亡率在各区分布不均。部分远郊区县慢性病过早死亡率相对较高，可能由于慢性病早期筛查与干预不足、居民健康意识相对薄弱、基层医疗服务对慢性病管理能力有限等因素导致慢性病病情控制不佳，进而过早死亡风险增加。

北京市各区在人均平均寿命、慢性病发病率、健康满意度及慢性病过早死亡率等健康相关指标上存在显著空间差异。为改善全市居民健康状况，应加强远郊区县医疗资源投入与布局优化，提升基层医疗服务能力，加强慢性病早期筛查与综合管理；同时，针对不同区域特点开展健康教育与健康促进活动，以实现区域健康均衡发展。

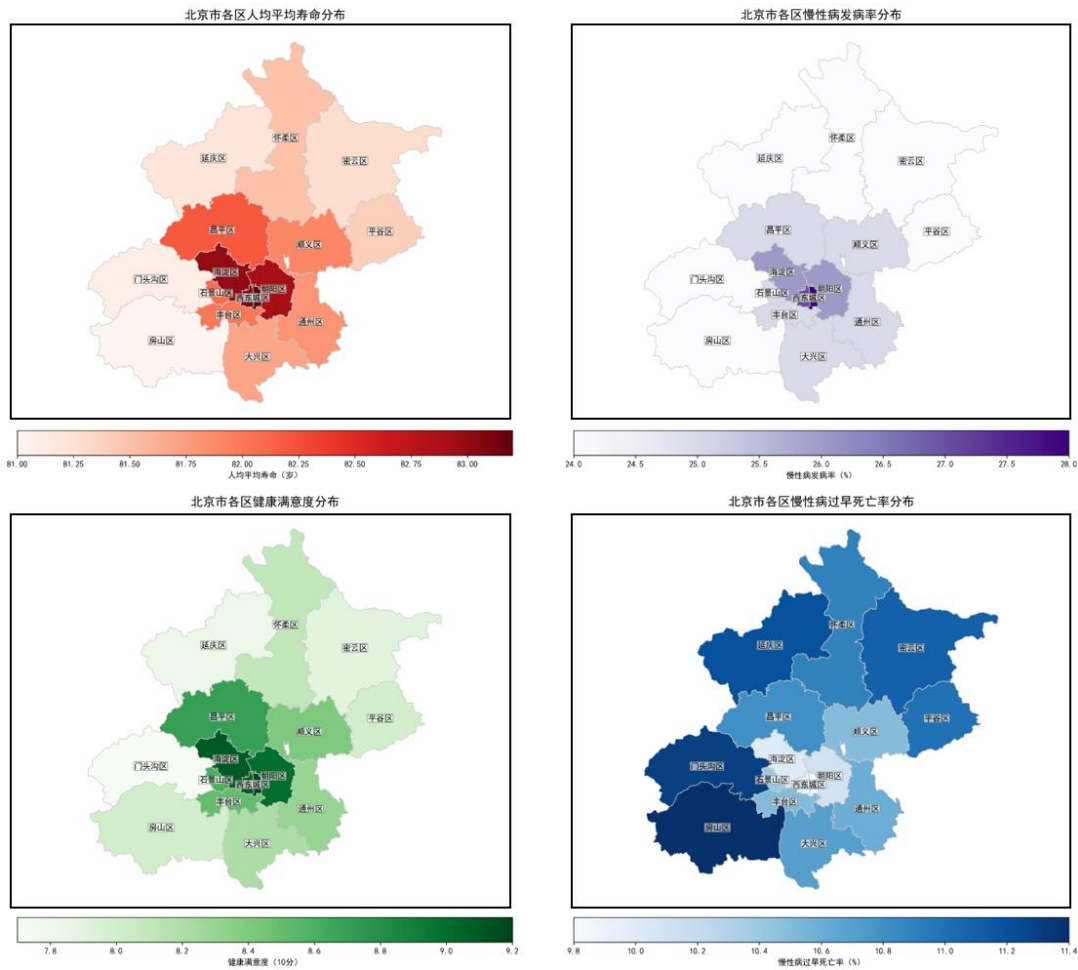


图 4.2 北京市各区居民健康状况数据分布图

4.1.2 康养资源指数与居民健康指数模型构建

(1) 康养资源与居民健康状况指标相关性分析

首先根据康养资源的数量指标与居民健康状况指标进行 Spearman 相关性分析，从相关性矩阵来看，医疗设施数量、公园绿地数量和文化设施数量与居民的人均平均寿命、慢性病发病率以及健康满意度都呈现出显著的正相关性，与慢性病过早死亡率呈现出显著的负相关性。这表明这些资源的丰富程度在很大程度上影响着居民的健康状况。而养老机构数量与各项健康指标的相关性较弱，推测可能是养老机构的服务质量、覆盖人群等方面存在问题，或者是养老机构在整体康养体系中的作用相对有限。

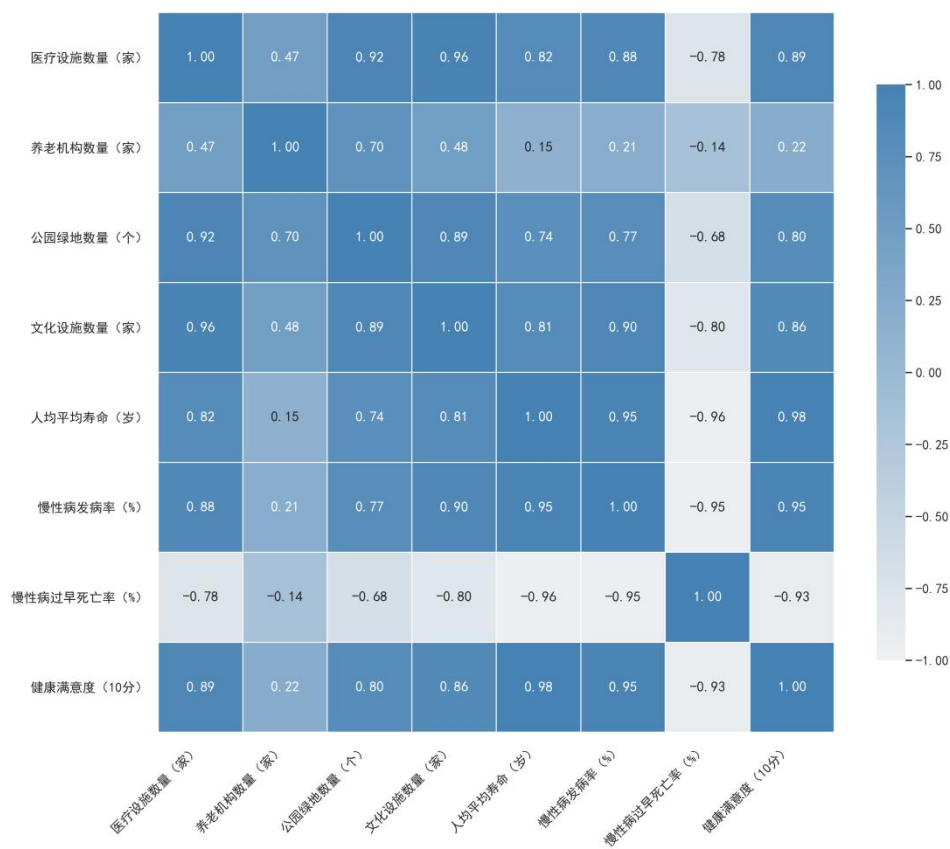


图 4.3 北京市各区康养资源与居民健康状况相关性热力图

(2) 康养指数模型

由于资源种类较多，同时考虑到空间上北京市各区的面积及常驻人口，因此直接比较康养资源数量指标和居民健康状况指标意义不明显，需要将多种资源进行整合，统一评估指标，构建康养资源指数模型，以衡量某一地区的康养资源供给能力。基于前期相关性分析结果，将养老机构数量这一作用有限的指标进行剔除，选择康养资源与居民健康状况相关性强的指标构建康养资源指数。

团队考虑到北京市各区如果按照面积来进行康养资源分配并不合理，因为北京市各区面积不平衡，以昌平区、东城区和西城区为例，昌平区常驻人口约 227 万人，东城区和西城区合计常驻人口 181 万人，然而昌平区面积约为 1344 平方千米，东城区和西城

区合计面积约为 92.54 平方千米，因此如果按照各区面积进行分配会导致严重的资源分配不平衡，影响后续的评价与优化工作，而按照人口进行分配是较为合适的方案。

因此，我们定义康养资源指数为 R_i ，计算公式如下：

$$R_i = \frac{\alpha_1 \cdot M_i + \alpha_2 \cdot E_i + \alpha_3 \cdot G_i + \alpha_4 \cdot C_i}{P_i}$$

其中： R_i 为康养资源指数； M_i 为医疗设施数量（家）； E_i 为区域 i 的养老机构数（家）； G_i 为区域 i 的公园绿地数量（个）； C_i 为区域 i 的文化设施数； P_i 为区域内人口数量（万人）。

公式的意义在于每类康养资源与人口之间形成分布密度，从而突出服务压力；权重 α_1 到 α_4 能够反映不同类型康养资源的影响强度，各权重之和为 1。

本次针对该康养指数模型的 α_1 到 α_4 的确定，采用熵值法进行设定。熵值法是一种客观赋权法，源于热力学中的熵概念，后被引入数学和统计学领域用于多指标综合评价。该方法通过计算指标的熵值来判断指标的离散程度，熵值越小，指标的离散程度越大，说明该指标提供的信息量越多，在综合评价中所起的作用越大，从而可通过熵值确定各指标的权重。在经济学、管理学、生态学等领域的数据分析和评价中具有广泛应用。

熵值法的具体计算步骤为：数据标准化；计算指标的比重矩阵；计算指标的熵值 e_j ；计算指标的差异系数 d_j ；计算指标权重 w_j 。

根据熵值法计算指标重要度，医疗设施数量权重 22.87%，养老机构数量权重 20.719%，公园绿地数量权重 24.454%，文化设施数量权重 31.957%。

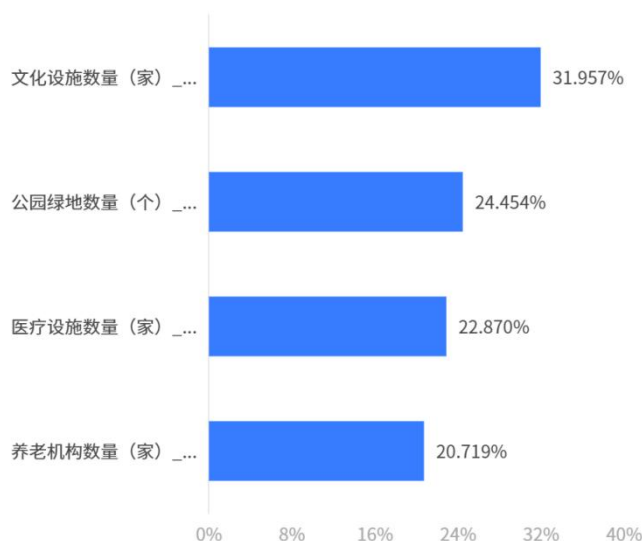


图 4.4 康养资源指标重要度直方图

因此最终确定北京市各区康养指数模型为：

$$R_i = \frac{0.2287 \cdot M_i + 0.20719 \cdot E_i + 0.24454 \cdot G_i + 0.31957 \cdot C_i}{P_i}$$

其中： R_i 为康养资源指数； M_i 为医疗设施数量（家）； E_i 为区域 i 的养老机构数（家）； G_i 为区域 i 的公园绿地数量（个）； C_i 为区域 i 的文化设施。

（3）居民健康指数模型

本次收集居民健康数据指标为人均平均寿命（岁），慢性病发病率（%），慢性病过早死亡率（%）以及居民健康满意度（10 分制）。居民健康指数需要多指标综合计算求得，因此构建居民健康指数模型，计算公式如下：

$$H_i = \beta_1 \cdot L_i - \beta_2 \cdot D_i - \beta_3 \cdot D_i + \beta_4 \cdot S_i$$

其中：Hi 为居民健康指数；Li 为人均平均寿命（岁）；Di 为慢性病发病率（%）；慢性病过早死亡率（%）；Si 为居民健康满意度（10 分制）。

公式的意义在于每类居民健康数据指标进行综合，得到一个反映居民健康状况的总体指数；权重 β_1 到 β_4 能够反映不同类型康养资源的影响强度，各权重之和为 1。

本次针对该康养指数模型的 β_1 到 β_4 的确定，采用熵值法进行设定。根据熵值法计算指标重要度，人均平均寿命权重为 21.653%，慢性病发病率权重为 44.534%，慢性病过早死亡率权重为 15.374%，居民健康满意度权重为 18.44%。

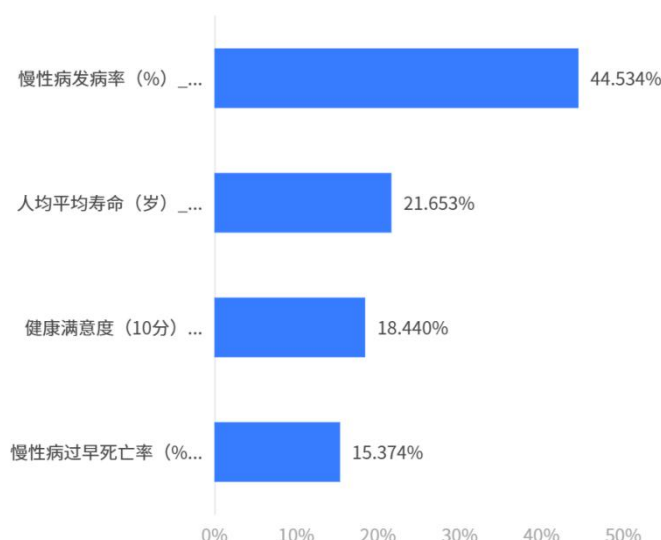


图 4.5 康养资源指标重要度直方图

因此最终确定北京市各区居民健康指数模型为：

$$H_i = 0.21653 \cdot L_i - 0.44534 \cdot D_i - 0.15374 \cdot D_{ti} + 0.1844 \cdot S_i$$

其中：Hi 为居民健康指数；Li 为人均平均寿命（岁）；Di 为慢性病发病率（%）；Dti 慢性病过早死亡率（%）；Si 为居民健康满意度（10 分制）。

4.1.3 资源分布合理性定量评价模型构建

基于康养资源指数与居民健康指数模型，计算相应指数，如下表所示。团队为了研究所得的康养资源指数与居民健康指数的关系，采用 Pearson 相关性分析法分析两者之间的关系。

表 4.3 康养资源指数与居民健康指数结果表

区	康养资源指数	居民健康指数
东城区	4.237648588	5.735604
西城区	2.6935625	6.125477
朝阳区	1.26030238	6.478323
丰台区	1.338523123	6.57509
石景山区	2.675785461	6.630557
海淀区	1.2816688	6.53379
门头沟区	1.847015152	6.555041
房山区	1.321728659	6.573334
通州区	1.025530081	6.47953
顺义区	1.167284366	6.534997
昌平区	0.947116417	6.609154
大兴区	1.146716015	6.424063
怀柔区	2.432264172	6.776909
平谷区	1.993591247	6.721442
密云区	1.658478178	6.665975
延庆区	2.260256647	6.610508

结果发现，北京区各区康养资源指数与居民健康指数整体呈负相关，相关系数为 -0.6008，p 值为 0.0138。分析原因为北京区存在两个特殊区，东城区和西城区，他们的康养资源指数相对高、但居民健康指数相对低，导致整体分析呈现负相关关系。分别来看，东城区和西城区康养资源与居民健康指数呈负相关，而其他区呈正相关。说明东城区和西城区的居民健康指数受到慢性病发病率的影响较大，导致居民健康指数相对较低。

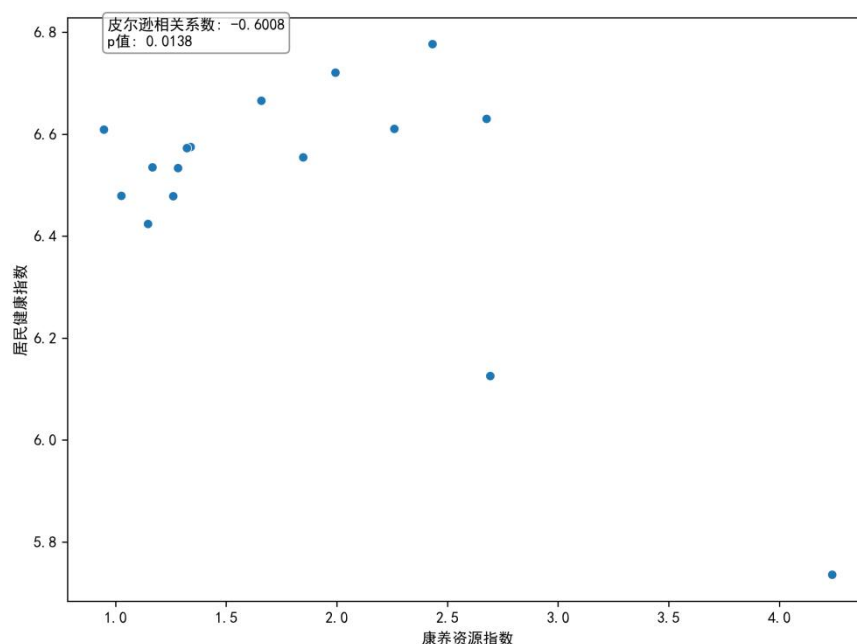


图 4.6 康养资源指数与居民健康指数交会图

筛除东城区和西城区，其他 14 区康养资源指数与居民健康指数总体上呈正相关关系，相关系数为 0.6834，p 值为 0.0035。

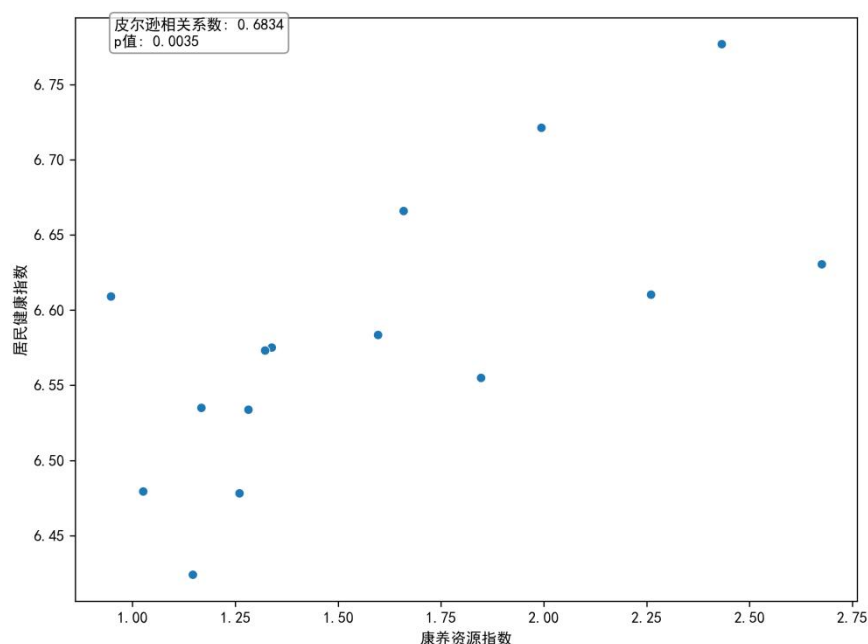


图 4.7 康养资源指数与居民健康指数交会图（去除东城区和西城区）

北京市各区总体相关系数-0.6008，筛除特殊区后相关性系数为 0.6834，但与 1 相差较大，同时各区相关性的正负，以及与 1 的偏差程度表明北京市康养资源供给与居民健康状况相对不符，存在资源配置不合理的情况。

同时，为了进一步探究资源分配的均衡性，基于康养资源指数，探究其离散程度。引入康养资源方差：

$$\sigma_R^2 = \frac{1}{n} \sum_{i=1}^n (R_i - \bar{R})^2$$

其中， σ_R^2 为康养资源方差； R_i 为康养资源指数； $\bar{R}$ 为康养资源指数均值。

经计算方差为 0.7075，方差相对较大，因此判断北京市各区当前康养资源分配不合理。

4.1.4 资源分配优化模型构建

针对北京市各区康养资源分配不合理的情况，首先建立资源量评估模型，评估北京市整体资源，模型如下：

$$PR_i = \frac{R_i}{H_i}$$

其中 PR_i 为北京市各区资源匹配度， R_i 为康养资源指数， H_i 为居民健康指数。

根据北京市各区资源匹配度频数直方图，北京市资源匹配度 PR_i 小于 1，主要集中在

在 0.4 以下，说明人均资源分配不足的现象较为普遍，因此解决资源不均衡的问题转变为在资源总量不变的条件下，调度资源，使得资源分布更均衡更合理的优化问题。

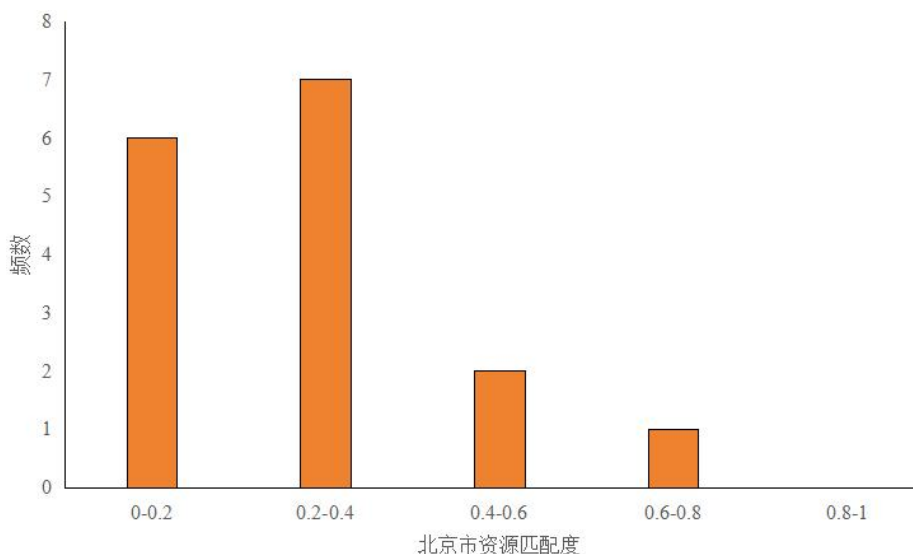


图 4.8 北京市各区资源匹配度频数直方图

本资源分配优化模型主要以最小化康养资源指数方差和北京市各区医疗设施居民可达时间为目标，并给予他们各自相应的权重 a 和 b ， a 和 b 的和为 1，约束条件为在康养资源量不变的条件下重新配置康养资源，以达到提升康养服务公平性和可达性的目标。设计的优化目标函数如下：

$$\min \left(a \cdot \sigma_R^2 + b \cdot \frac{1}{n} \sum_{i=1}^n T_i \right)$$

$$\sum_{i=1}^n R_i^{\text{优化后}} = \sum_{i=1}^n R_i^{\text{优化前}}$$

其中， a 和 b 为调节服务公平性和可达性的权重系数，和为 1，约束条件为康养资源总量不变。

随后针对该优化问题，采用 SLSQP 优化方法。SLSQP（Sequential Least Squares Programming，序列最小二乘规划法）是一种求解非线性约束优化问题的迭代算法，核心思想是通过将非线性问题转化为一系列二次规划（QP）子问题来逐步逼近最优解。尤其适合中小规模、连续变量、光滑目标函数的场景。具有鲁棒性强的优点，对目标函数和约束的非线性程度有较好的适应性，尤其适合目标函数光滑（可导）的场景；同时能够保证收敛性，在合理假设下（如约束规范、目标函数凸性等），能收敛到局部最优解（若目标函数为凸函数，则可收敛到全局最优）；并且严格保证约束条件，通过迭代强制满足约束条件，避免优化过程中出现不可行解。

在本次构建的优化问题中，约束条件明确，总康养资源指数不变（等式约束），SLSQP 能直接处理此类约束；目标函数可导，方差和均值函数均为光滑函数，满足 SLSQP 对目标函数可导的要求；并且是小规模问题，变量维度为 16（北京市 16 个区），SLSQP

的计算成本可控。因此，对于该问题 SLSQP 方法能有效平衡康养资源指数方差和平均可达时间，同时严格保持总资源不变。最终绘制损失函数曲线和优化前后康养指数对比图，由于康养指数为按照人口密度的结果，因此需要换算为康养资源量来说明本次各区资源量配置。

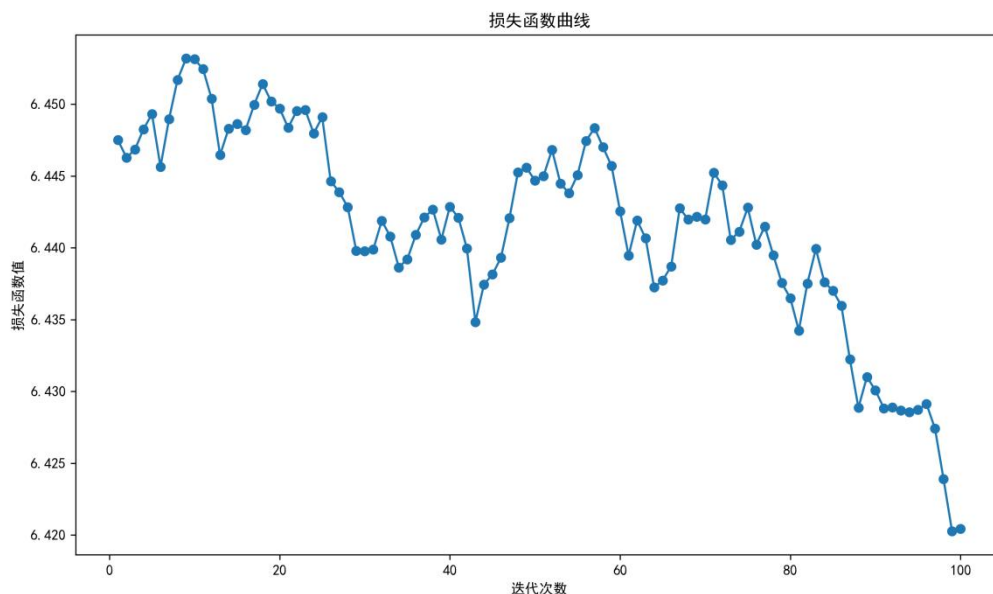


图 4.9 损失函数曲线

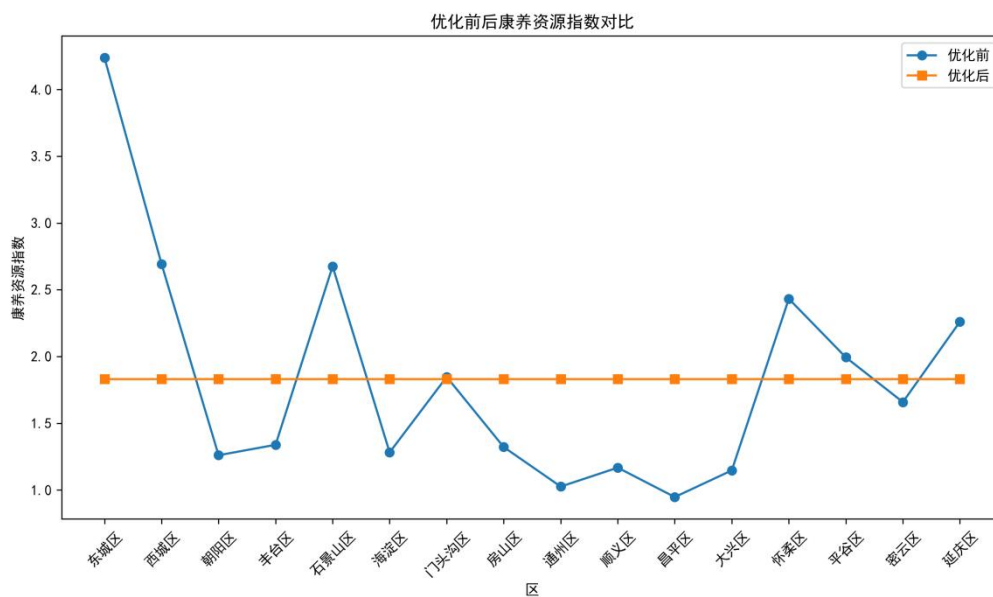


图 4.10 优化前后康养资源指数对比

根据人口换算康养资源量，如下图所示。在总资源量不变的条件下，优化康养资源，要适当减少东城区、西城区、石景山区的康养资源量；同时提高朝阳区、丰台区、海淀区、房山区、通州区、顺义区、昌平区和大兴区的康养资源量；门头沟区、怀柔区、平谷区、密云区和延庆区康养资源量暂时不需要进行调配，以此解决北京市各区康养资源分配不均衡的问题，提升康养服务的公平性和可达性。

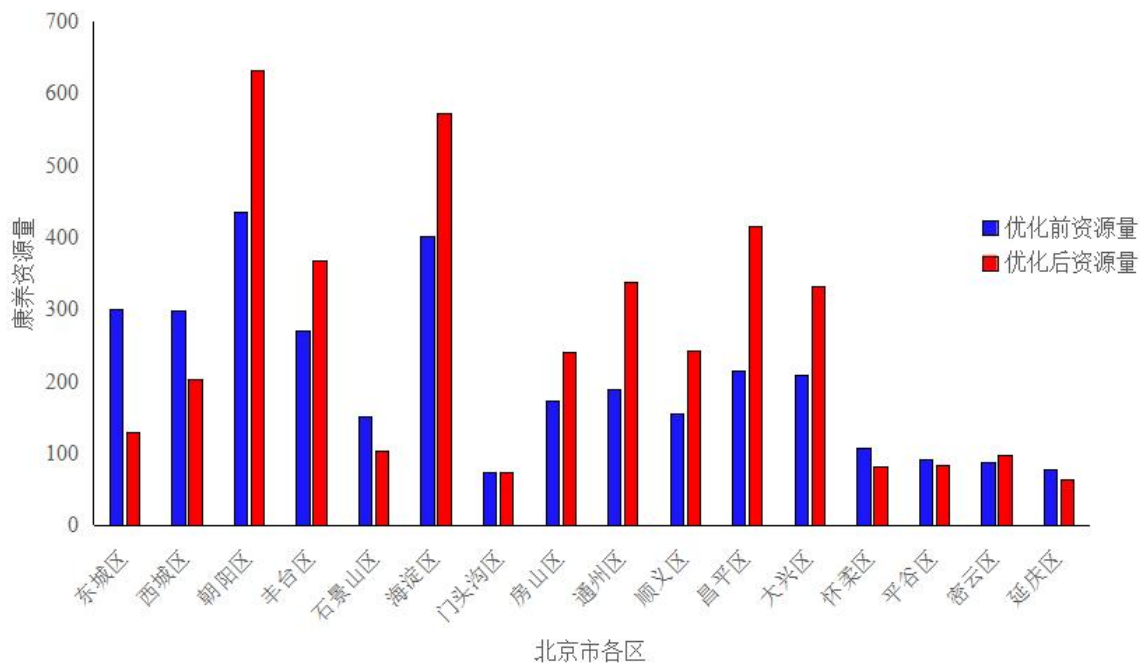


图 4.11 优化前后康养资源指数对比

4.2 问题二模型建立与求解

本研究采用“指标构建-权重确定-综合评价-合理指导”的技术路线。首先基于研究目标与数据可得性构建指标体系，然后运用熵权法确定各指标客观权重，再通过 TOPSIS 法计算各区域相对贴近度，最终结合空间分析方法揭示区域发展格局特征。最后采用强化学习给出指导性的意见。

4.2.1 问题求解思路

针对康养城市发展水平的综合评价问题，本文从康养资源丰富度、居民健康状况、环境质量以及经济发展水平四个维度构建评价体系，并选取相关定量指标。首先对各指标数据进行标准化处理，消除量纲影响；其次采用熵权法计算指标权重，确保权重分配客观合理；然后应用 TOPSIS 方法计算各区域的贴近度，得到康养发展水平的排序结果；最后结合区域分布格局，提出基于强化学习的动态优化改进方案，为康养城市建设提供科学指导。

4.2.2 指标体系构建

在构建康养城市综合评价模型前，从康养资源、居民健康、环境质量和经济发展四个维度出发，选取具有代表性的定量指标。具体指标体系如下表所示：

表 4.4 不同评价维度下选取的关键指标

维度	指标	指标含义	性质
康养资源 丰富度	医疗设施数量	衡量资源供给能 力和公平性	正向
	每千人床位数		
	养老机构数量		
	公园绿地数量		
	文化设施数量		
居民健康 状况	平均寿命	衡量居民整体健 康水平	平均寿命、满意度为正向；发 病率、过早死亡率为反向
	慢性病发病率		
	慢性病过早死亡率		
	健康满意度		
环境质量	PM2.5 年均浓度	反映生态环境质 量与健康风险	反向
经济发展 水平	地区生产总值	保障康养服务可 持续发展的经济	正向
	人均可支配收入	基础	

医疗设施数量：反映区域医疗服务供给规模，数量越多说明基础医疗资源越充足，直接影响居民就医便利性与可及性。

每千人床位数：衡量区域医疗资源人均占有水平，通过床位数与人口的配比，体现医疗服务保障能力的强弱。

养老机构数量：体现区域为老年人提供专业化养老服务的机构规模，数量多少关系到养老服务供给的充足程度。

公园绿地数量：代表区域生态休闲空间的供给规模，数量越多说明居民可享受的户外休闲资源越丰富。

文化设施数量：体现区域文化服务供给能力，数量多少反映文化资源的丰富度与居民文化生活的便利程度。

平均寿命：综合衡量区域居民健康水平与生活质量，受医疗条件、环境质量、生活水平等多因素影响。

慢性病发病率：反映区域居民慢性病患病情况，发病率高低与生活习惯、环境因素及医疗预防水平相关。

慢性病过早死亡率：体现区域慢性病防治成效，死亡率高低反映对重大疾病的早期干预与治疗能力。

健康满意度：通过居民主观评价反映区域健康保障水平，体现医疗服务与健康管理的综合效果。

PM2.5 年均浓度：衡量区域空气质量的核心指标，浓度高低直接关联居民呼吸系统健康与生态环境质量。

地区生产总值（GDP）：衡量区域经济总体规模与发展水平，是反映经济实力和产业发展状况的核心指标。

人均可支配收入：体现区域居民平均收入水平与消费能力，直接关联生活质量与区域经济发展成果共享程度。

4.2.3 基于熵权 TOPSIS 法的评价模型

（1）数据标准化

首先，将部分数值指标转化为更为合理的密度分布表示。为了由于各指标的量纲和取值范围不同，需对原始数据进行标准化处理。采用极差标准化方法，将所有指标转换为[0,1]区间内的无量纲数值。对于正向指标，标准化公式为：

$$Z_{ij} = \frac{X_{ij} - \min(X_j)}{\max(X_j) - \min(X_j)}$$

其中， X_{ij} 表示原始值， Z_{ij} 表示归一化后的值， $\min$ 和 $\max$ 分别表示最大和最小值。

对于正向指标，标准化公式为：

$$Z_{ij} = \frac{\max(X_j) - X_{ij}}{\max(X_j) - \min(X_j)}$$

其中， X_{ij} 为第 i 区域在第 j 个指标上的原始值， Z_{ij} 为标准化后的值。

（2）权重确定

目前，关于指标权重的确定方法，大体可分为两个方面，即主观赋权法和客观赋权法。主观赋权法通过倚重较为威望的专家经验来评价各指标的重要性，并据此确立各指标权重，有效规避了算术平均法在权重分配上可能出现的偏差。但其过分依赖于评价者及专家的个人判断，可能会削弱结果的客观性。而在客观赋权法中，则是基于指标的实际数据，采纳数学手段计算而得到指标权重，其侧重点在于通过综合评定各指标的变异程度和对整体指标集的影响，从而得出权重值。由于客观赋权法通常以数学理论为基础，因此结果可信度高，保证客观真实。熵法是一种客观赋权方法，其基本原理是根据指标数据的离散程度确定权重。指标的信息熵越小，表明其变异程度越大，提供的信息越多，权重也就越高。具体计算步骤如下：

计算第 j 个指标在第 i 个区域的比重：

$$p_{ij} = \frac{Z_{ij}}{\sum_{i=1}^m Z_{ij}}$$

计算第 j 个指标的信息熵：

$$e_j = -k \sum_{i=1}^m p_{ij} \ln(p_{ij}), k = \frac{1}{\ln(m)}$$

计算第 j 个指标的熵权：

$$w_j = \frac{1 - e_j}{\sum_{j=1}^n (1 - e_j)}$$

表 4.5 不同指标权重值

维度	指标	权重
康养资源丰 富度	医疗设施分布密度（家 / 平方千米）	0.157365299
	养老机构分布密度（家 / 平方千米）	0.106056751
	公园绿地分布密度（个 / 平方千米）	0.141079613
	文化设施分布密度（家 / 平方千米）	0.17140914
居民健康状 况	每千人床位数	0.075230365
	人均平均寿命（岁）	0.040819346
	慢性病发病率（%）	0.014844305
	健康满意度（10 分）	0.034762787
环境质量	慢性病过早死亡率（%）	0.0323844
	PM2.5 年均浓度（ $\mu\text{g}/\text{m}^3$ ）	0.025092553
	地区生产总值（亿元）	0.09186512
经济发展水 平	人均可支配收入（元）	0.045814071
	年常住人口（万人）	0.06327625

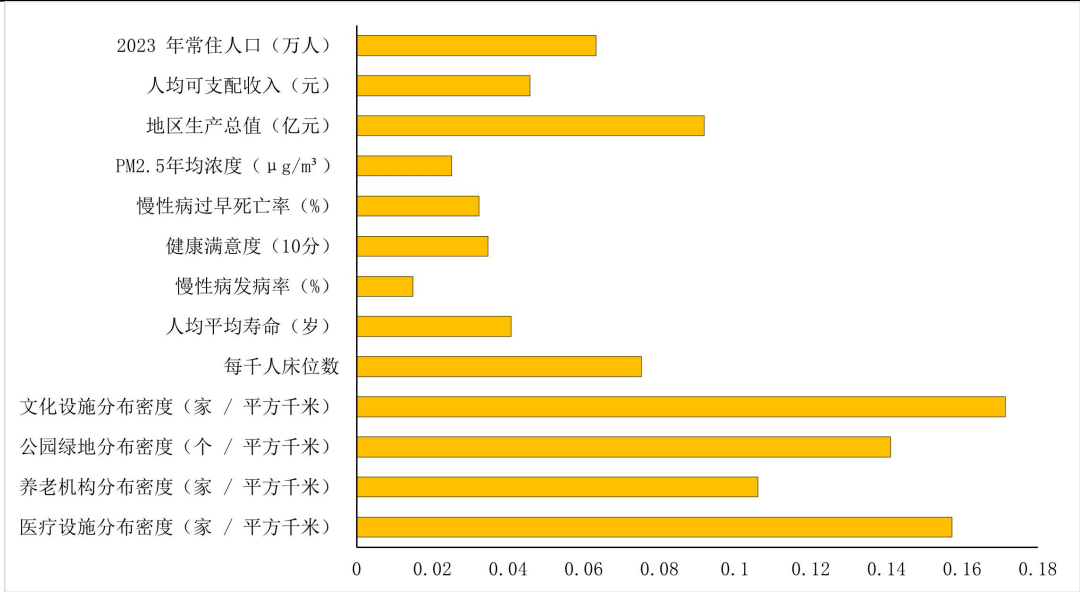


图 4.12 不同指标权重分布柱状图

这张指标权重柱状图来看，各指标对康养城市发展水平的影响程度差异明显。其中文化设施权重最高，养老床位数、公园绿地、养老机构、医疗设施等指标权重也相对较高，反映出医疗、养老及生态环境在康养城市建设里的关键地位，充足的养老床位与机构、密集的医疗设施能为居民提供可靠的健康保障，而公园绿地则关乎居住环境质量和居民身心健康。

相比之下，PM2.5 年均浓度、慢性病发病率、健康满意度、慢性病过早死亡率（%）人均平均寿命（岁）等与居民健康直接相关指标的权重相对较低，可能是由于这些指标不能直接影响康养城市的发展水平。其余指标权重也不高，表明在此次康养城市发展水

平评价中，经济和人口规模并非主导因素，更侧重于养老资源与环境等方面对康养的支撑。

(3) TOPSIS 法

在完成数据标准化和权重确定后，采用 TOPSIS（Technique for Order Preference by Similarity to Ideal Solution）方法对各区域的康养发展水平进行综合评价^[3,4]。TOPSIS 综合评价法是一种接近理想值的排序方法，适用于存在多个评价对象、多个指标的情景。在计算过程中，需要首先确定每个指标的理想解和负理想解，即确定各项指标在全部选定方案中的最优值和最劣值。然后计算各个方案与理想解、负理想解之间的距离，以此确定各个方案与最优方案的接近程度，从而实现对不同决策方案的评价比较。其计算步骤如下：

建立决策化矩阵，对于 m 个评价对象， n 个评价指标，决策矩阵为：

$$X = (x_{ij})_{m \times n}$$

构建加权标准化矩阵：通过决策矩阵与权重相乘得到：

$$V = (w_j \cdot x_{ij})_{m \times n}$$

确定理想解和负理想解：针对正向型指标的计算公式如下，如存在负向型指标，则计算相反：

$$A^+ = \{\max(v_{1j}, v_{2j}, \dots, v_{mj}) | j = 1, 2, \dots, n\}$$

$$A^- = \{\min(v_{1j}, v_{2j}, \dots, v_{mj}) | j = 1, 2, \dots, n\}$$

其中， A^+ 表示理想解， A^- 表示负理想解。

计算各方案到理想解和负理想解的距离：

$$D_i^+ = \sqrt{\sum_{j=1}^m (v_{ij} - v_j^+)^2}$$

$$D_i^- = \sqrt{\sum_{j=1}^m (v_{ij} - v_j^-)^2}$$

其中， D_i^+ 表示第 i 个对象与理想解的欧氏距离， D_i^- 表示第 i 个对象与负理想解的欧氏距离。

计算贴近度：值越大，表示该区越接近理想解，康养发展水平越高。

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}$$

表 4.6 北京市不同地区康养发展水平综合评价得分

区名	综合得分	康养资源得分	居民健康状 况得分	环境质量得分	经济发展水平 得分	排名
东城区	0.82	0.95	0.88	0.86	0.34	1
西城区	0.75	0.82	0.86	0.71	0.48	2
海淀区	0.50	0.29	0.86	0.71	0.96	3
朝阳区	0.42	0.20	0.81	0.57	0.89	4
石景山区	0.36	0.39	0.59	0.43	0.14	5
丰台区	0.27	0.20	0.54	0.29	0.35	6
昌平区	0.17	0.04	0.56	0.43	0.31	7
通州区	0.15	0.05	0.46	0.14	0.28	8
大兴区	0.14	0.03	0.41	0.00	0.36	9
顺义区	0.14	0.04	0.51	0.29	0.23	10
怀柔区	0.08	0.01	0.35	0.57	0.06	11
平谷区	0.07	0.01	0.30	0.71	0.05	12
密云区	0.06	0.00	0.25	0.86	0.05	13
房山区	0.06	0.02	0.18	0.14	0.13	14
延庆区	0.05	0.00	0.20	1.00	0.02	15
门头沟区	0.04	0.00	0.15	0.86	0.01	16

4.2.4 区域发展水平评价及分布格局

通过绘制康养城市综合得分分布图可以看出，北京市康养发展水平有待提高，大部分城市综合得分较低，部分综合得分在 0.3~0.5，少部分得分在 0.7 以上，呈现两级风化情况。

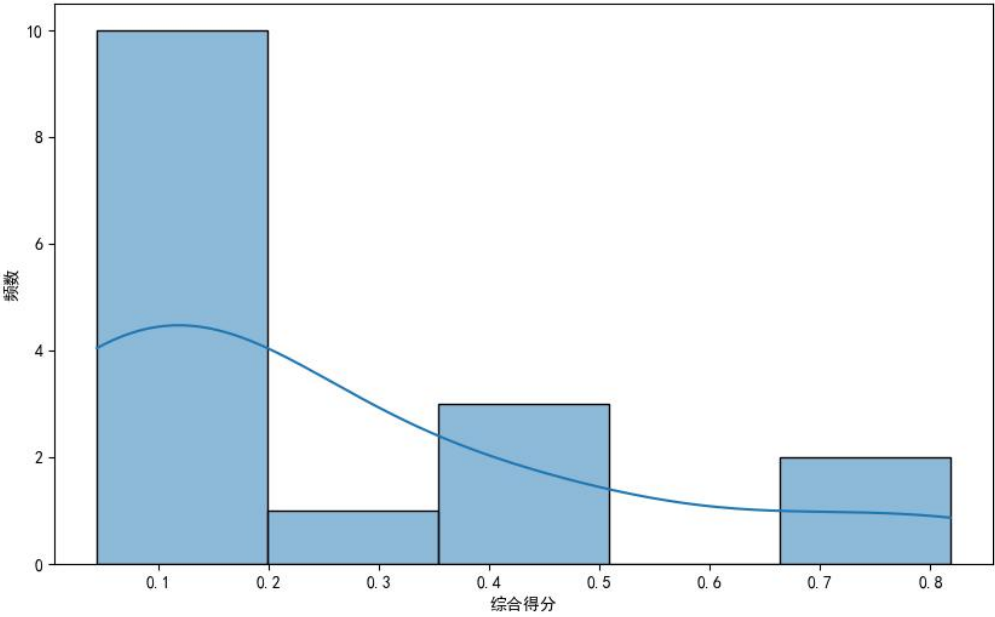


图 4.13 康养城市综合得分分布直方图

从康养综合得分与各个评价维度的相关性热力图可以看出，决定北京市总体康养发展水平的关键因素为康养资源和居民健康状况，这些因素直接决定了北京市的康养发展水平。其次为经济发展水平，经济接影响着以上两种的发展。

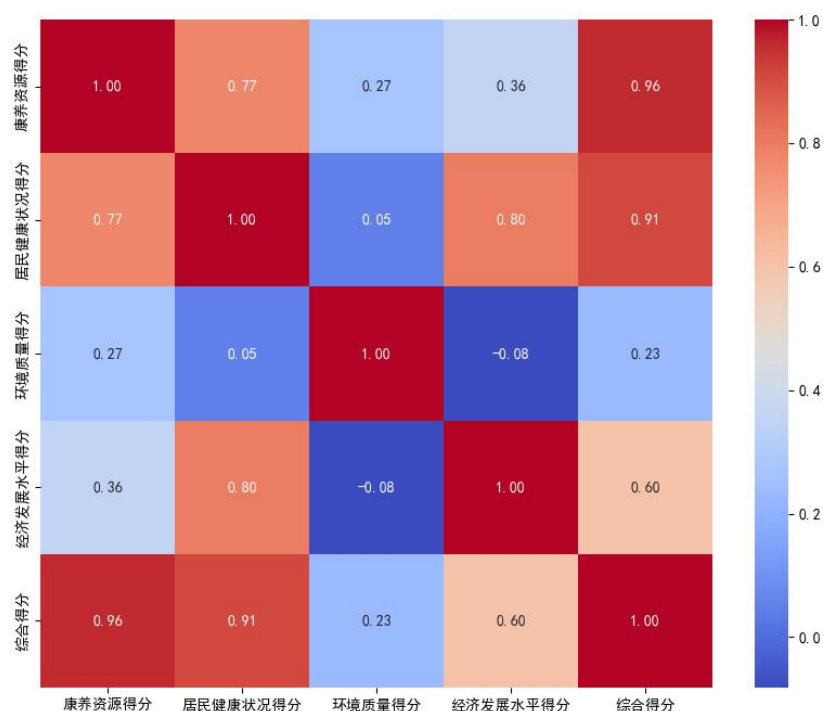


图 4.14 不同评价维度相关性热力图

进一步，根据康养资源得分和居民健康状况两个关键指标，运用聚类方法，将北京市不同区康养发展水平分为了发展水平低、发展水平中等、发展水平较好三个等级，其中怀柔、平谷、密云、房山、延庆、门头沟发展水平较低；石景山、丰台、昌平、通州、大兴、顺义发展水平中等；东城区、西城区、海淀区、朝阳区发展相对较好。

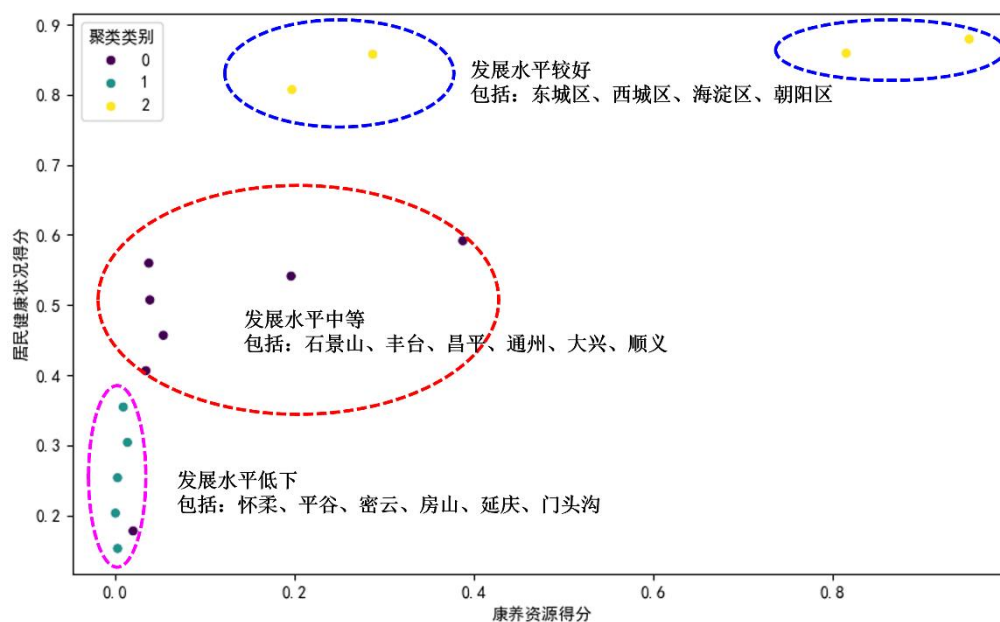


图 4.15 聚类结果及康养水平分级图

在“北京市各区康养资源得分热力地图”中，海淀区、朝阳区等城区康养资源得分相对较高。这些区域集中了大量的医疗设施、养老机构等康养资源，能够为居民提供较为丰富的康养服务选择。相比之下，远郊区县如平谷区、密云区等康养资源得分较低，资源相对匮乏。这种分布格局与城市的发展规划、人口分布以及资源配置政策密切相关。核心城区人口密集，经济发达，吸引了更多的康养资源集聚；而远郊区县由于地理区位、经济发展水平等因素的限制，康养资源投入相对不足，导致资源分布不均衡。

依据“北京市各区居民健康状况得分热力地图”，海淀区、东城区等区域居民健康状况得分较高。这些区域居民平均寿命相对较长，慢性病发病率较低，反映出其在医疗卫生服务、健康管理等方面取得了较好的成效。而延庆区、密云区等部分远郊区县居民健康状况得分较低，可能受到医疗资源相对不足、居民健康意识以及生活环境等多种因素的综合影响。这种差异提示在康养建设中，需关注不同区域居民健康状况的特点，针对性地加强健康服务体系建设，提高居民整体健康水平。

从“北京市各区环境质量得分热力地图”可知，延庆区、密云区等远郊区县环境质量得分较高，这些区域生态环境良好，PM2.5 年均浓度较低，拥有丰富的自然生态资源，为康养产业发展提供了良好的自然基础。而丰台区、大兴区等部分城区环境质量得分相对较低，可能受到城市建设、工业发展等因素的影响，环境压力较大。环境质量在康养建设中具有重要作用，良好的环境有助于提升居民的康养体验和生活质量，因此在康养建设规划中，应注重不同区域环境质量的差异，充分发挥生态优势区域的康养潜力，同时加强环境质量相对薄弱区域的生态治理与保护。

观察“北京市各区经济发展水平得分热力地图”，海淀区、朝阳区经济发展水平得分较高，强大的经济实力为康养建设提供了坚实的物质基础，能够在康养设施建设、服务创新等方面投入更多资源。而延庆区、平谷区等远郊区县经济发展水平得分较低，经济发展的相对滞后在一定程度上限制了康养建设的投入与发展速度。经济发展水平与康养建设紧密相关，较高的经济水平能够支撑康养产业的多元化发展，提升康养服务的质量和水平；反之，经济薄弱区域在康养建设中面临资源短缺等困境，需要通过政策引导、区域协作等方式，促进康养建设与经济发展的良性互动。

从“北京市各区综合得分热力地图”来看，北京市康养建设综合发展水平在区域间呈现出明显的差异。东城区、西城区等核心城区综合得分相对较高，处于热力地图的暖色区域，表明这些区域的康养建设综合发展水平处于领先地位。这得益于其在医疗、养老、经济等多方面的资源优势与长期积累的发展基础。而延庆区、密云区等远郊区县综合得分较低，处于冷色区域，说明其康养建设综合发展水平相对滞后。总体呈现出以核心城区为中心，向周边远郊区县逐渐递减的空间分布格局，反映出北京市康养建设在空间上存在不均衡性。

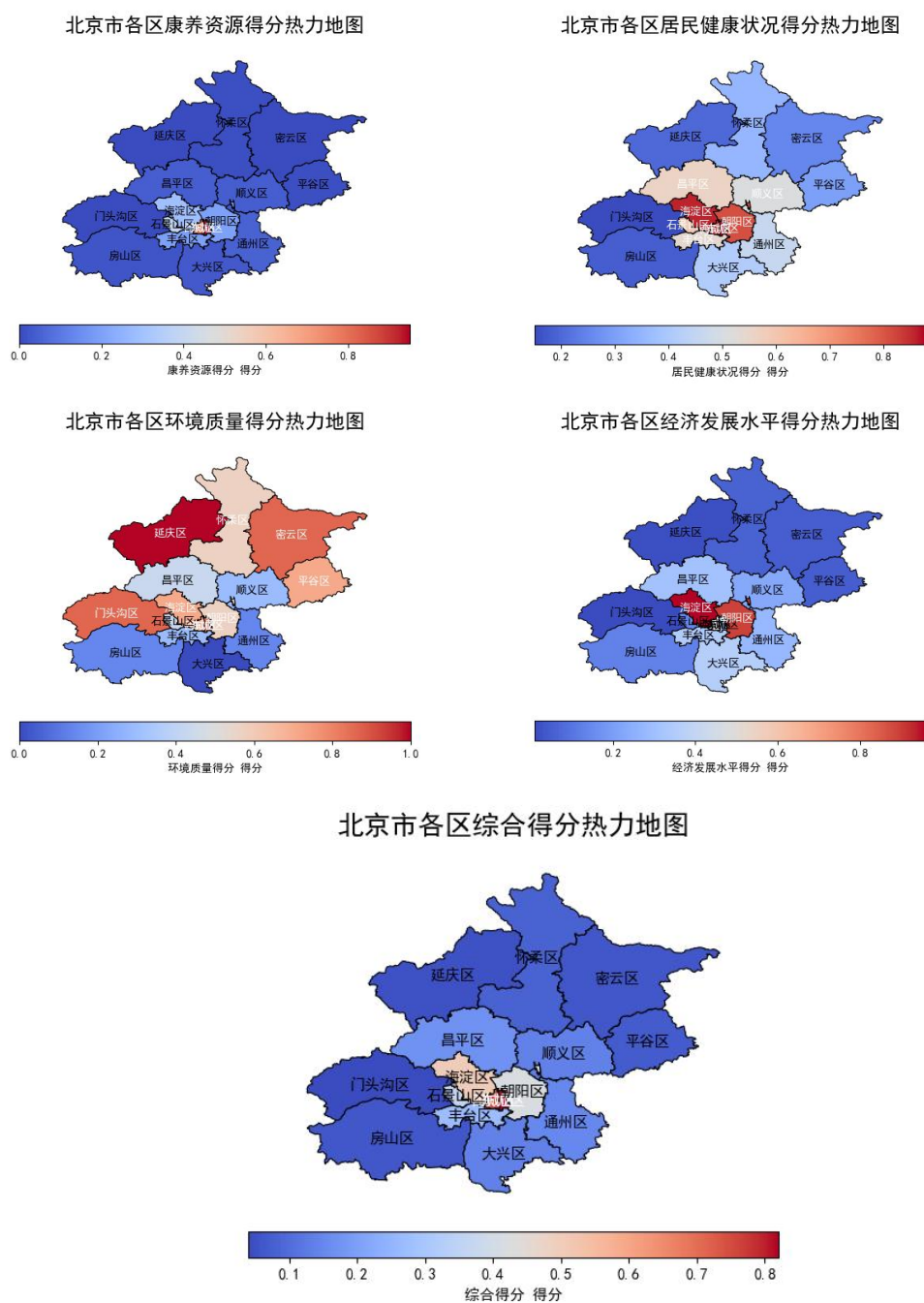


图 4.16 不同评价维度得分分布热力图

4.2.5 基于强化学习的改进方案

为提出定量化改进意见，进一步收集了 2018-2022 年康养指标数据，构建包含多维动态状态空间的马尔可夫决策过程，借助 PPO 强化学习算法^[5,6]；以 2023 年数据作为初始状态，通过模拟未来的规划，将过去六年历史数据隐含的发展规律融入状态空间构建；利用 2018-2023 年数据分布特征初始化策略网络，通过折扣回报强化对长期趋势的学习，最终使生成的优化策略既能基于历史数据规律，又能适应未来的动态变化。

(1) PPO 算法

Proximal Policy Optimization (PPO) 算法是强化学习中一种高效的策略梯度方法，PPO 通过引入剪切概率比 (Clipped Surrogate Objective) 替代传统策略梯度，显著提高了训练的稳定性与收敛速度。PPO 属于基于策略的近端优化算法，其目标是通过迭代优化，学习一个最优策略，以在给定状态下输出最优动作，最大化期望累计奖励。

在本文中，PPO 被应用于康养城市发展水平优化问题，通过调整各区域的医疗资源、养老设施、绿地分布等多维康养资源配置（即动作），以提升基于 TOPSIS 模型计算的康养发展水平得分（即奖励），实现康养城市长期健康可持续发展目标。

策略 (Policy)：从状态到动作的映射关系，由神经网络参数化。

(2) 环境与状态空间设计

在康养城市优化场景中，PPO 算法将康养发展规划建模为马尔可夫决策过程 (MDP)：

状态 (State)：城市各康养指标的当前状态（如医疗设施密度、公园绿地密度等）。

动作 (Action)：对各指标的调整策略（如增加医疗设施）。

奖励 (Reward)：基于康养综合得分的反馈信号。

(3) 网络结构

包括策略网络和价值网络。

策略网络：输入层为标准化状态向量（包含医疗设施密度、养老机构密度等康养指标），再通过两层两层全连接层，每层 128 个神经元，激活函数为 ReLU；输出为动作均值向量（对应各指标的调整比例）和对数标准差向量。

价值网络：输入层为标准化状态向量（包含医疗设施密度、养老机构密度等康养指标），再通过两层两层全连接层，每层 128 个神经元，激活函数为 ReLU；输出为状态价值估计。

(4) 损失函数设计

PPO-Clip 损失函数：PPO 的核心创新是通过裁剪策略比率来限制参数更新幅度，避免策略震荡或崩溃。其数学表达式为：

$$L^{CLIP}(\theta) = E_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]$$

其中， $r_t(\theta) = \frac{\pi_\theta(a_t|S_t)}{\pi_{\theta/d}(a_t|S_t)}$ 是新旧策略的概率比； $A_t = G_t - V(s_t)$ 是优势函数，表示动作价值相对于状态价值的优势； $\epsilon=0.2$ 是裁剪阈值，控制策略更新的最大幅度； $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$ 将比率裁剪到 $[0.8, 1.2]$ 范围内。

价值函数损失，通过均方误差最小化状态价值估计与实际回报的差异：

$$L^V(\phi) = E_t[(V_\phi(s_t) - G_t)^2]$$

其中 $V_\phi(s_t)$ 是价值网络预测的状态价值； $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$ 是从时间步 t 开始的折扣累积回报； $\gamma = 0.99$ 是折扣因子，控制未来奖励的重要性。

(5) 超参数设置

表 4.7 超参数设置表

参数	取值	作用说明
γ （折扣因子）	0.99	平衡短期与长期奖励，使算法更关注长期康养水平提升
Σ （裁剪阈值）	0.2	限制策略更新幅度，防止因数据分布变化导致策略崩塌
k_epochs（更新轮次）	10	对同一批数据多次优化，提高样本利用效率
entropy_coef（熵系数）	0.01	控制策略探索性，避免过早陷入局部最优
hidden_dim（隐藏层维度）	128	决定网络表达能力，128 维在计算效率和拟合能力间取得平衡
lr（学习率）	3e-4	控制参数更新步长，Adam 优化器自适应调整

（6）训练结果

Episode 奖励在第 500 轮后趋于稳定，平均奖励达到 1.82，策略损失和价值损失在迭代过程中逐渐收敛，并趋于稳定，表明模型达到了较好的效果。

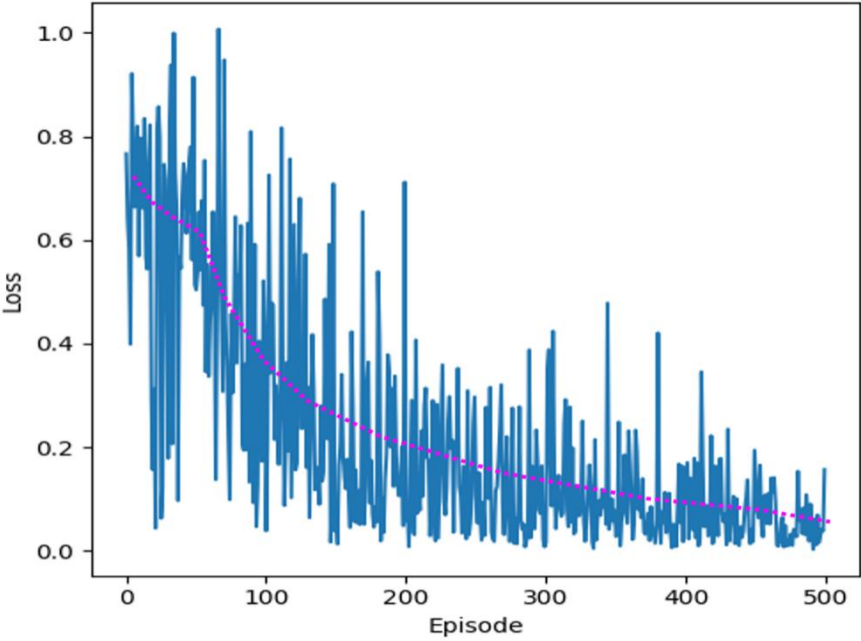
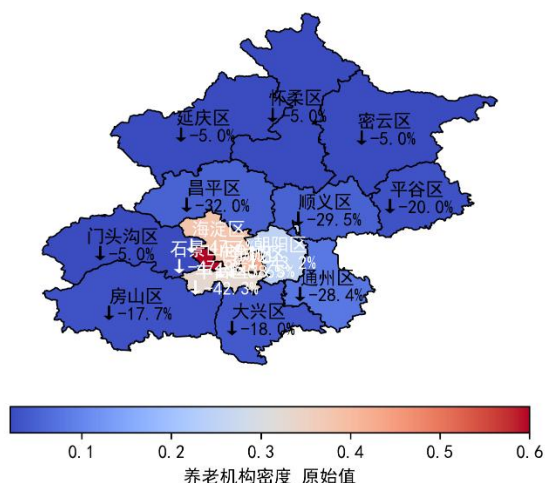


图 4.17 损失函数下降图

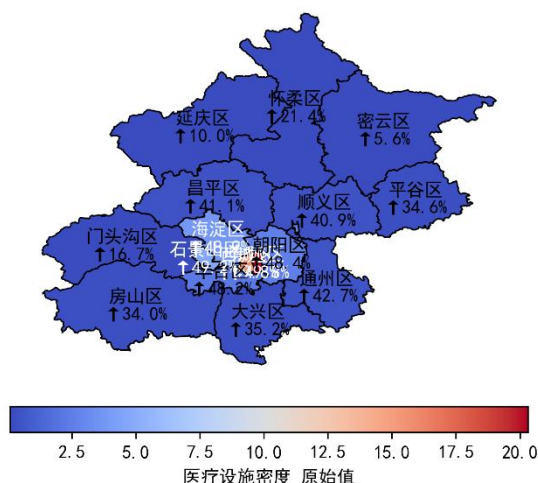
（7）改进建议

从各城区的情况来看，康养城市优化建议在整体上都有所调整。在医疗养老资源配置方面，多数城区会增加医疗设施密度与每千人床位数，提升医疗服务的可及性与承载能力，同时大多会降低养老机构密度，对现有养老机构进行整合优化以提高运营效率；生态环境与健康促进领域，各城区普遍增加公园绿地密度来改善生态，虽提高慢性病发病率和慢性病过早死亡率数值，但意在精准监测疾病、加强防控，还通过提升医疗服务质量和健康宣传等提升居民健康满意度；文化与经济发展层面，多数城区降低文化设施密度以整合资源、提高利用率，同时提高地区生产总值和人均可支配收入，为康养建设提供经济支撑并提升居民康养消费能力；在环境质量方面，基于实际环境监测与治理情况做出的合理调整，以保障环境质量的持续改善。以下为各指标在现有的基础进行调整的百分比，根据该定量调整指标可有效的提升北京市康养建设的发展。

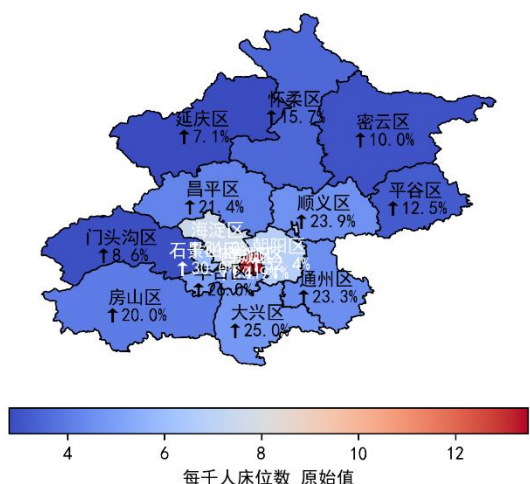
北京市各区养老机构密度热力地图



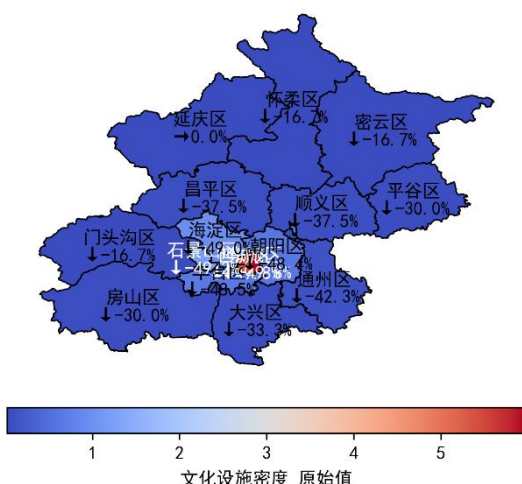
北京市各区医疗设施密度热力地图



北京市各区每千人床位数热力地图



北京市各区文化设施密度热力地图



北京市各区公园绿地密度热力地图

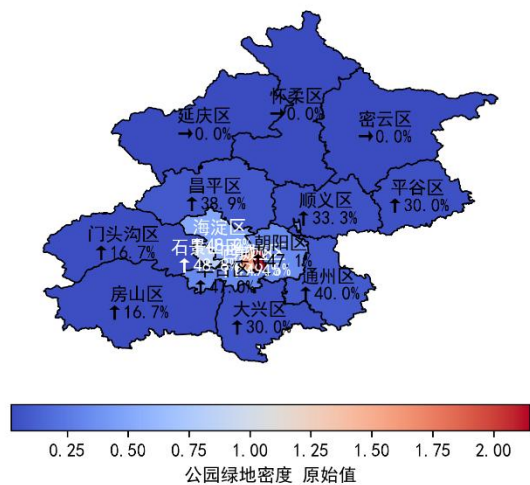


图 4.18 不同地区康养发展调整具体指标分布图

4.3 问题三模型建立与求解

4.3.1 问题三求解思路

问题三本质为多目标优化问题。首先，先明确问题，应平衡研究城市各区最大化服务覆盖率、资源利用效率和最小化成本这三个目标，考虑设施选址等空间约束。构建模型时，确定决策变量，设好最大化覆盖率、效率和最小化成本的目标函数，服务半径、预算等约束条件。在多目标优化方法选择上，拟采用粒子群优化（PSO）、差分进化算法（DE）、第三代非支配排序遗传算法（NSGAIII）三种前沿优化算法对结果进行分析总结^[7-10]。最后依据三种算法模型结果与问题一结果进行对比分析合理的提出具体的实施策略，包括政策建议、资金投入方向、技术创新措施等。

4.3.2 问题三模型变量定义

首先从实际的城市角度进行建模，依据北京市十六个行政区划原则进行编号，以及将城市中预设的康养设施位置点进行定义如下：

- 北京市下辖区划分为 16 个行政区域，编号为 $i=1, 2, 3, 4, \dots, 16$;
- 城市中拟设置 m 个可选康养设施位置点，编号为 $j=1, 2, 3, \dots, m$;
- 其他基本变量定义如下表：

表 4.8 基本参数变量定义说明

符号	含义
D_i	第 i 区域的老年人口需求
C_j	第 j 设施建设成本
S_j	第 j 设施最大服务容量
d_{ij}	第 i 区域设施到 j 设施点距离
r_j	最大服务半径
B	投资预算
$x_j \in \{0,1\}$	是否在 j 点建设设施
$y_{ij} \in \{0,1\}$	区域 i 是否由设施 j 服务

本模型目标是最大化康养服务的覆盖率和资源利用效率，同时最小化成本。由此目标函数定义如下：

最大化加权服务覆盖率定义如下：

$$\max f_1 = \sum_{i=1}^n w_i \cdot \min \left(1, \sum_{j=1}^m y_{ij} \right)$$

w_i 为区域 i 的需求权重（本题基于慢性病发病率）。

最大化资源利用效率定义如下：

$$\max f_2 = \sum_{j=1}^m \frac{\text{设施 } j \text{ 实际服务人数}}{\text{设施 } j \text{ 最大容量}}$$

假设建设成本为区 GDP 的 5%，最小化总成本定义如下：

$$\min f_3 = \sum_{j=1}^m \left(c_j^{\text{建设}} + c_j^{\text{运营}} \cdot x_j \right)$$

由于设施提供的服务必须在可达的范围内，若某设施点超过其服务半径则不可服务该区域，且建设设施支出不可超过总的成本，以及服务具有唯一性的条件约束。本模型的约束条件公式定义如下：

假设最大服务半径为 6 公里，服务半径约束：

$$d_{ij} \leq r_{\max} \cdot x_j \quad (d_{ij} \text{ 为区域 } i \text{ 到设施 } j \text{ 的距离})$$

预算约束：

$$\sum_{j=1}^m c_j^{\text{建设}} \cdot x_j \leq B$$

假设每个区域仅可由一个主服务设施覆盖，则对应约束：

$$y_{ij} \leq x_j, \text{ 且 } \sum_{j=1}^m y_{ij} = 1$$

上述变量定义结束后,为使得模型求解结果更加合理,避免单一算法的偏差,拟采用粒子群优化(PSO)、差分进化算法(DE)、第三代改进的非支配排序遗传算法(NSGAIII)三种前沿优化算法进行建模优化求解。优化康养资源分配。

4.3.3 多目标粒子群优化(PSO)模型建立及求解

粒子群优化算法受鸟群觅食行为启发。在求解康养资源优化配置问题时,它通过粒子在解空间中的飞行来寻找最优解,每个粒子代表一种可能的设施选址方案。而且,它在一定程度上能够快速收敛到一个较好的解,在短时间内为决策者提供初步的参考方案。在考虑到康养资源优化配置问题涉及的设施候选点数量(即决策变量维度)较少,区域数量也不是特别庞大时,PSO算法能够展现出较高的计算效率。它可以在相对较少的迭代次数内找到较优解,从而节省计算时间和计算资源。

结合上述定义的目标函数及约束函数,在原始PSO模型基础上引入动态权重调整策略,根据迭代次数自适应调整目标优先级,前期侧重覆盖率,后期侧重成本,以更好地平衡不同阶段的优化重点。同时,通过罚函数将服务半径、预算等约束融入适应度计算,确保粒子群在搜索过程中生成的解满足约束条件。对惯性权重 w 、认知学习因子 C_1 、社会学习因子 C_2 、最大迭代次数 max_iter 和种群规模 pop_size 等关键参数进行研究和优化。通过不同参数组合的实验,分析参数对算法收敛速度和求解结果质量的影响,以确定最佳的参数设置,提高算法性能。PSO具体优化步骤设计如下:

(1) 数据准备及预处理

将收集到的北京市各行政区的数据、包括各区域的面积、医疗设施数量、养老机构数量、慢性病发病率等信息进行归一化处理。使其总和为1,以便在目标函数中合理体现各区域的需求程度。合理设置建设成本、运营成本、设施最大容量和区域到设施的距离矩阵等参数。建设成本根据区域经济情况(地区生产总值)设置,运营成本设置为固定范围,设施最大容量根据常住人口设置,距离矩阵根据区域地理位置简单模拟生成。

(2) 粒子群初始化

随机生成大小为 $\text{pop_size} * m$ 的粒子群位置矩阵,其中每个元素 $i \in \{0,1\}$ 表示是否在第 j 个。候选点建设设施,同时初始化速度矩阵 v 为全0矩阵。初始化个体最优位置矩阵 $pbest$ 为当前粒子群位置矩阵的副本,个体最优适应度向量 $pbest_fitness$ 初始化为正无穷。

随机选择一个粒子作为全局最优位置 $gbest$,并计算其适应度作为全局最优适应度 $gbest_fitness$ 。

(3) 适应度计算

定义适应度函数 $\text{fitness_fountion}(x)$,用于计算每个粒子(设施选址方案)的适应度。在函数内部,分别计算三个目标函数 f_1 、 f_2 和 f_3 的值,并根据设定的权重组合得到适应度值。如果方案不满足约束条件(如没有设施建设或超出预算),则添加惩罚项。对粒子群中的每个粒子,调用适应度函数计算其适应度,并与个体最优适应度进行比较。若当

前适应度更优，则更新个体最优位置和个体最优适应度。同时，比较当前粒子的适应度与全局最优适应度，若更优则更新全局最优位置和全局最优适应度。

（4）粒子位置和速度更新

在每次迭代中，根据公式：

$$v = w * v + c_1 * r_1 * (pbest - x) + c_2 * r_2 * (gbest - x)$$

更新粒子的速度，其中 r_1 和 r_2 是在[0,1]区间内随机生成的矩阵。根据公式：

$$x = \text{np. clip}(x + v, 0, 1). \text{astype(int)}$$

更新粒子的位置，并将位置值限制在 0 和 1 之间（表示是否建设设施的决策）。

（5）迭代终止条件判断

检查是否达到最大迭代次数 max_iter 。如果达到，则停止迭代，输出全局最优位置 $gbest$ 作为最优设施选址方案，以及全局最优适应度 $gbest_fitness$ 作为最优目标函数值。如果未达到最大迭代次数，则返回步骤 3 继续进行下一次迭代。

（6）结果分析与评估

在基于 Python 编程的 PSO 算法优化过程中，团队通过增加粒子群多样性，避免陷入局部最优，通过优化，PSO 模型最终参数设置如下：

表 4.9 PSO 模型参数设置

符号	含义	值
n_particles	粒子数量	15
max_iter	最大迭代次数	100
w	惯性权重	0.7
c1	个体学习因子	0.25
c2	社会学习因子	0.5

在经过 100 轮次迭代优化过程中，最优适应度与 PSO 算法收敛曲线逐步趋于拟合，说明算法最终达到较好的分析结果。其可视化如下：

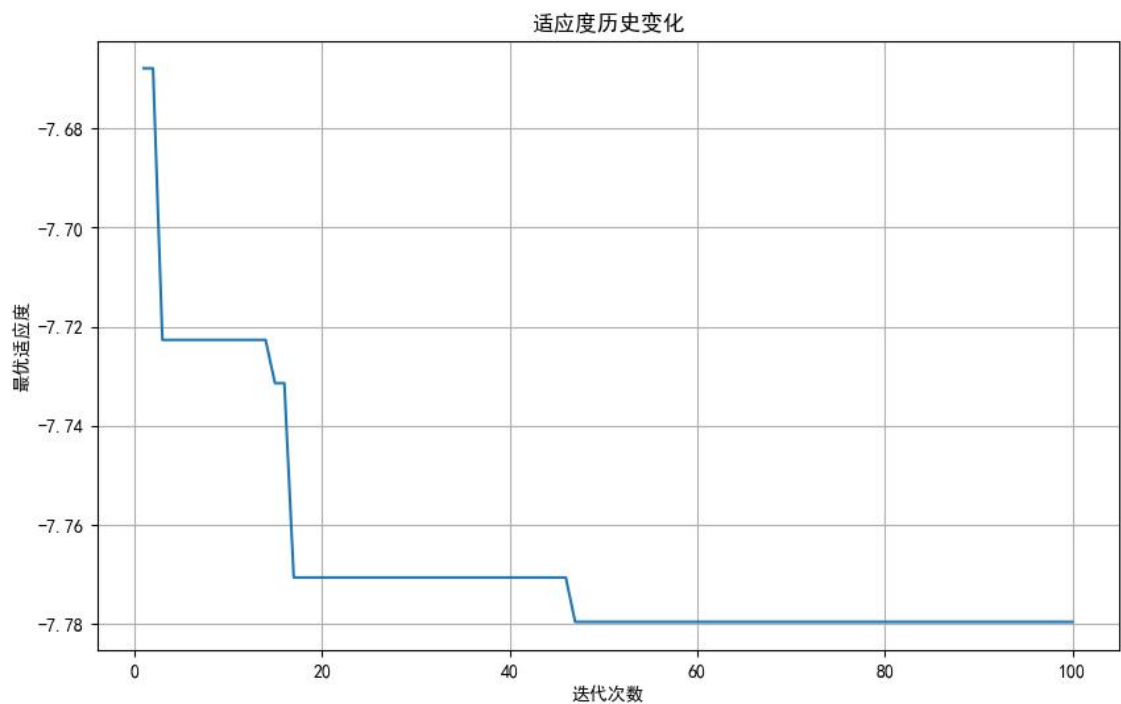


图 4.19 PSO 算法适应度历史变化图

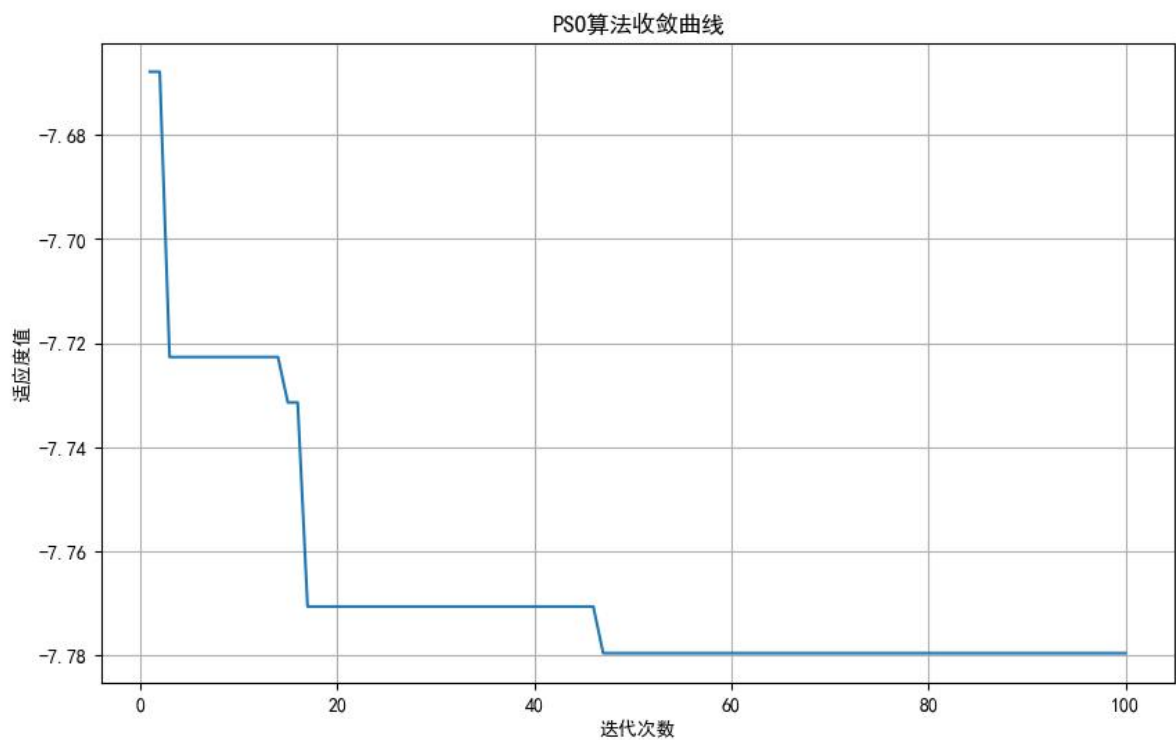


图 4.20 PSO 算法收敛曲线

基于上述目标函数及约束条件经过 PSO 算法优化，北京市优化后提升空间最大的五个行政区分别为海淀区、房山区、平谷区、密云区、延庆区，优化后各区可最大化康养服务的覆盖率和资源利用效率，同时最小化成本。

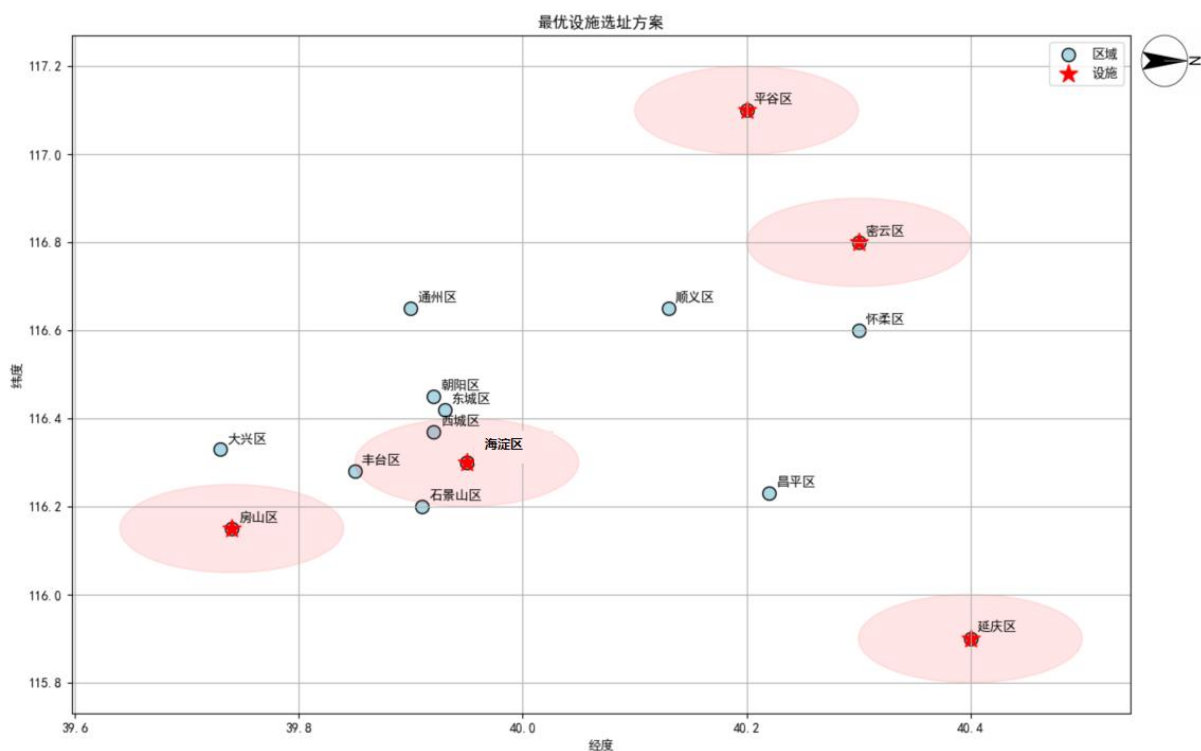


图 4.21 最优设施改进空间行政区

上述五区具体改进设施方案如下表：

表 4.10 各区设施改进方案设置

区名	医疗设施	养老机构	公园绿地	文化设施
海淀区	应增加 146 家	应减少 3 家	应增加 29 个	应增加 60 家
房山区	应增加 3111 家	应增加 146 家	应增加 389 个	应增加 962 家
平谷区	应增加 1406 家	应增加 62 家	应增加 156 家	应增加 438 家
密云区	应增加 3560 家	应增加 163 家	应增加 414 个	应增 1044 家
延庆区	应增加 3133 家	应增加 148 家	应增加 364 个	应增加 938 家

4.3.4 差分进化算法（DE）模型建立及求解

差分进化算法是基于种群的全局优化算法，具有较强的全局搜索能力。它通过差分变异和交叉操作生成新的个体，这种操作方式能够在解空间中广泛探索不同的可能性，有助于避免算法陷入局部最优。在康养资源优化配置问题中，由于目标函数可能较为复杂，存在多个局部最优解，DE 算法的全局搜索特性可以更好地找到全局最优解或者接近全局最优的解。

当问题的决策变量维度较高，例如区域数量众多或者设施候选点数量很多时，DE算法能够在高维解空间中保持较好的搜索性能。它可以有效地处理复杂的约束条件和目标函数，在高维空间中找到较优的设施选址方案。

差分进化算法具体优化步骤设计如下：

(1) 模型参数设置

设定种群规模： pop_size （如：80）、最大迭代次数 max_iter (如：300)、缩放因子 F (如 0.8)和交叉概率 CR (如：0.7)等关键参数。

(2) 种群初始化

随机生成大小为 $pop_size \times m$ 的初始种群矩阵 $population$ ，其中每个元素 $population_{ij} \in \{0,1\}$ 表示第 i 个个体（设施选址方案）是否在第 j 个候选点建设设施。同时，初始化全局最优解 $best_solution$ 和全局最优适应度 $best_fitness$ ，初始时将全局最优适应度设为正无穷大。

(3) 适应度计算

对每个个体 $population_i$ ，计算其三个目标函数值 f_1 （服务覆盖率）、 f_2 （资源利用效率）和 f_3 （总成本）。将多目标问题转化为单目标问题，通过加权组合的方式计算适应度值，初始设置为如下：

$$fitness = -5 \times f_1 - 3 \times f_2 + f_3 + penalty$$

其中， $penalty$ 是对违反约束条件（如预算约束、服务半径约束）的惩罚项。更新全局最优解和全局最优适应度。

(4) 差分进化操作

变异操作：对于每个个体 i ，从种群中随机选择三个不同的个体 r_1, r_2, r_3 ，生成变异向量：

$$v_i = population_{r_1} + F \times (population_{r_2} - population_{r_3})$$

其中， F 是缩放因子，控制差分的幅度。将变异向量的元素值限制在 $[0,1]$ 范围内，并进行二值化处理（大于 0.5 设为 1，否则设为 0）。

交叉操作：将变异向量 v_i 与当前个体 $population_i$ 进行交叉，生成试验向量 u_i 。对于每个维度 j ，以概率 CR 选择变异向量的值，以概率 $1 - CR$ 选择当前个体的值：

$$u_{ij} = \begin{cases} v_{ij}, & \text{if random}(0, 1) < CR \text{ or } j = j_{rand} \\ population_{ij}, & \text{otherwise} \end{cases}$$

其中， j_{rand} 是随机选择的一个维度，确保试验向量至少有一个维度来自变异向量。

(5) 选择操作

计算试验向量 u_i 的适应度值，并与当前个体 $population_i$ 的适应度值进行比较。如果试验向量的适应度更优，则用试验向量替换当前个体：

$$population_i = \begin{cases} u_i, & \text{if } fitness(u_i) < fitness(population_i) \\ population_i, & \text{otherwise} \end{cases}$$

同时，更新全局最优解和全局最优适应度。

(6) 迭代终止条件判断

检查是否达到最大迭代次数 `max_iter`。如果达到，则停止迭代，输出全局最优解 `best_solution` 作为最优设施选址方案，以及全局最优适应度 `best_fitness` 作为最优目标函数值。如果未达到最大迭代次数，则返回步骤 4 继续进行下一次迭代。

(7) 结果分析与评估

本算法基于 Python 语言，通过定义 `DEOptimizer` 类来定义差分进化算法，其具体参数设置如下：

表 4.11 DE 模型参数设置

符号	含义	值
<code>n_pop</code>	种群大小	50
<code>max_iter</code>	最大迭代次数	100
<code>F</code>	缩放因子	0.8
<code>CR</code>	交叉概率	0.9

算法适应度收敛过程如下，说明算法已抵达最终收敛，优化结束：

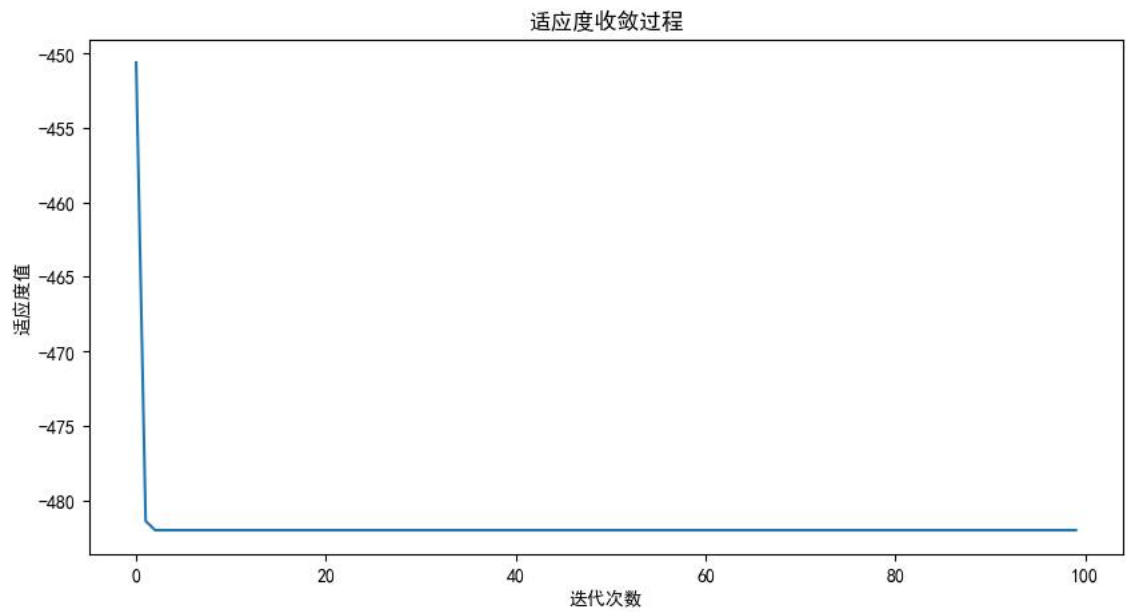


图 4.22 DE 算法适应度收敛过程

算法最终仅输出延庆区一个区的最优化设施改进方案如下：

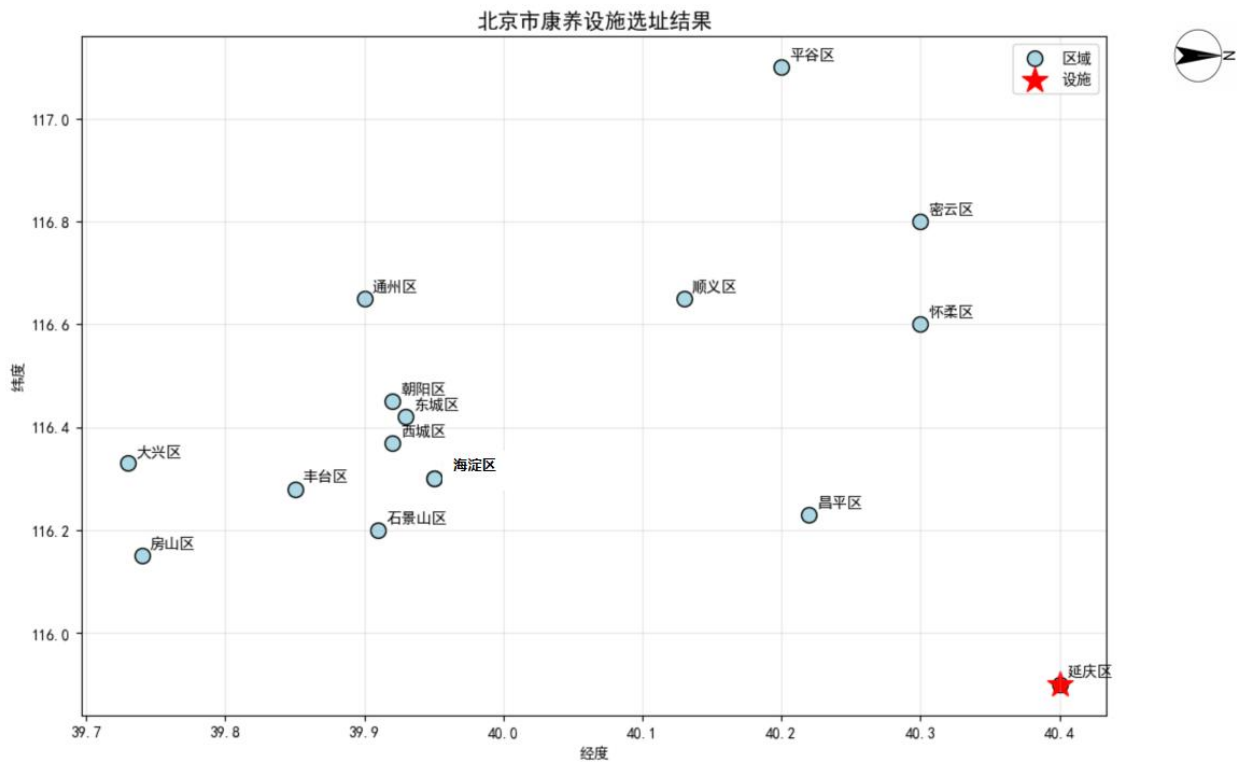


图 4.23 基于 DE 算法北京市康养设施建设改进区

延庆区最优养老设施及改进数量如下：

医疗设施：不合理，应增加 3266 家。具体举措：改善山区医疗条件，加强医疗卫生信息化建设。

养老机构：不合理，应增加 157 家。具体举措：发展乡村养老产业，促进养老服务均衡发展。

公园绿地：不合理，应增加 385 个。具体举措：保护生态环境，建设自然公园。

文化设施：不合理，应增加 997 家。具体举措：挖掘历史文化资源，打造文化旅游品牌。

4.3.5 NSGAIII 改进模型建立及求解

NSGA III（基于参考点的非支配遗传算法）是一种专门针对多目标优化问题的算法，NSGAIII 是 NSGAI 的改进版本，主要用于解决多目标优化问题。NSGAIII 通过引入参考点来保持种群的多样性和收敛性尤其适用于处理三个及以上目标的情况。在康养资源优化配置问题中，需要同时考虑服务覆盖率、资源利用效率和成本这三个重要目标，NSGA III 算法能够有效地处理这种多目标之间的权衡关系。它基于参考点的非支配排序方法，可以生成均匀分布的 Pareto 前沿，为决策者提供一系列具有代表性的最优解。

对于康养资源优化配置问题中存在的各种复杂约束条件，如服务半径约束、预算约

束等，NSGA III 算法能够在满足这些约束的前提下，找到最优的设施选址方案。它的分层优化和自适应参考点生成等改进策略，可以更好地应对复杂的问题结构和约束条件。NSGA III 算法优化求解过程如下：

初始化种群：随机生成初始设施选址方案，确保满足基本约束；迭代优化；非支配排序：将种群划分为不同等级的非支配解集；参考点关联：计算每个解与参考点的关联度；选择操作：基于参考点的拥挤度和关联度选择父代；交叉与变异：生成新解并应用约束处理规则；环境选择：合并父代和子代，保留精英解并更新种群；终止条件：达到最大迭代次数或收敛指标（如 Pareto 前沿变化小于阈值）。

基于 NSGA III 的改进设计如下：

改进一：参考点动态生成机制

初始参考点采用 DasDennis 方法生成，在三维目标空间(f_1, f_2, f_3)中均匀分布；迭代中每 50 代根据当前 Pareto 前沿极值动态调整参考点密度，在拥挤区域增加参考点数量，公式为：新参考点=当前极值 $\pm\delta$ ·标准差向量 ($\delta = 0.1$)

改进二：约束处理双层策略

第一层：可行性优先：可行解（满足所有硬性约束）直接进入非支配排序，不可行解按约束违反度降序排列；

第二层：动态罚函数：对不可行解引入罚项 $P = \sum_{k=1}^K \lambda_k \cdot \text{违反量}_k$ ， λ_k 随迭代次数从 10 线性衰减至 1，强制算法优先搜索可行域。

改进三：分层优化与精英保留

分层策略：前 100 代侧重优化 f_1 和 f_3 ，后 200 代加入 f_2 权重（权重系数从 0.3 增至 0.6）；

精英池机制：保留每代前 10% 最优解直接进入下一代，避免优质解丢失；

改进四：多样性增强技术

拥挤度距离改进：在三维空间中计算每个解到相邻解的曼哈顿距离，替代传统欧氏距离，提升分布均匀性；

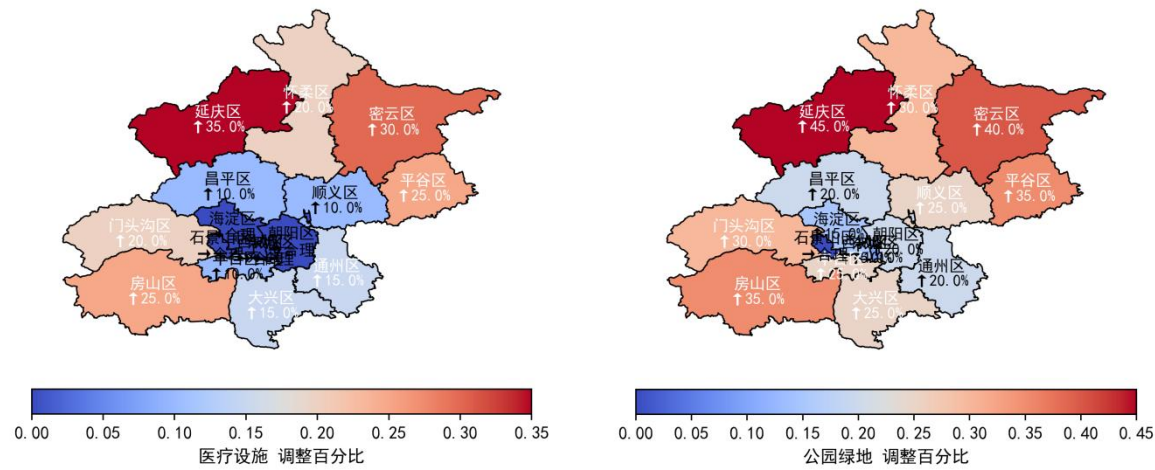
邻域扰动：对陷入局部最优的解随机翻转 12 个设施选址位 (x_j)，强制跳出局部极值。

改进模型最终输出优化方案如下：

表 4.12 各区设施改进方案设置

区名	医疗设施	养老机构	公园绿地	文化设施
东城区	合理	增加 5%	增加 10%	合理
西城区	合理	合理	增加 15%	合理
朝阳区	合理	合理	增加 20%	合理
丰台区	增加 10%	合理	增加 25%	合理
石景山	合理	合理	合理	合理
海淀区	合理	合理	增加 15%	合理
门头沟	增加 20%	增加 25%	增加 30%	增加 30%
房山区	增加 25%	增加 30%	增加 35%	增加 35%
通州区	增加 15%	合理	增加 20%	增加 20%
顺义区	增加 10%	合理	增加 25%	增加 25%
昌平区	增加 10%	合理	增加 20%	增加 20%
大兴区	增加 15%	合理	增加 25%	增加 25%
怀柔区	增加 20%	增加 25%	增加 30%	增加 30%
平谷区	增加 25%	增加 30%	增加 35%	增加 35%
密云区	增加 30%	增加 35%	增加 40%	增加 40%
延庆区	增加 35%	增加 40%	增加 45%	增加 45%

北京市各区医疗设施优化建议热力图 北京市各区公园绿地优化建议热力图



北京市各区养老机构优化建议热力图 北京市各区文化设施优化建议热力图

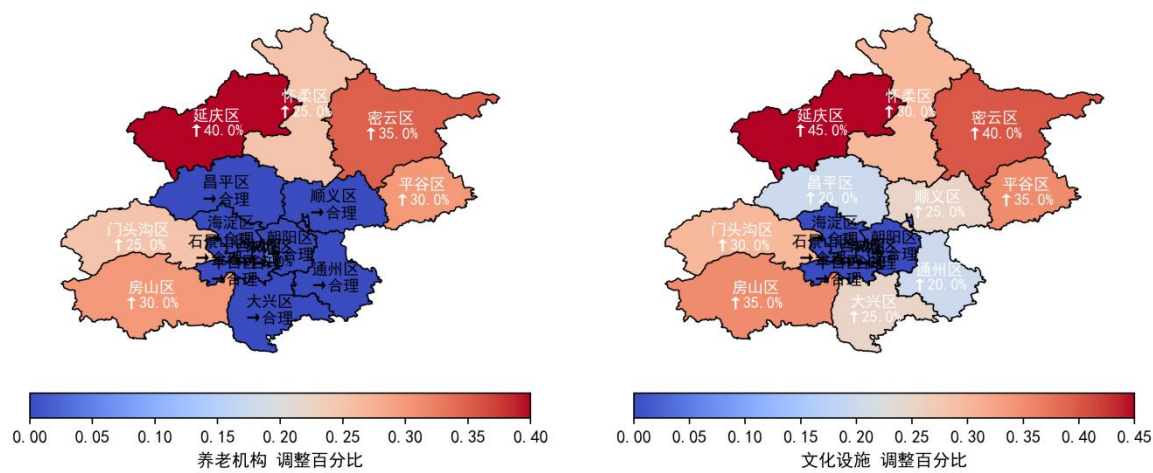


图 4.24 北京市各区多维资源优化建议图

4.3.6 模型分析与实施策略建议

结合模型结果和问题 1 的研究成果，具体的实施策略，包括政策建议、资金投入方向、技术创新措施如下：

(1) 模型分析

在问题三中，团队通过多目标优化模型对康养资源进行了优化配置，目标是最大化康养服务的覆盖率和资源利用效率，同时最小化成本。以下是基于上述三大模型结果的分析：

医疗设施：模型结果显示，部分区域如房山区、平谷区等需要大幅增加医疗设施数量。这表明这些区域的医疗资源相对不足，无法满足居民的康养需求。增加医疗设施不仅可以提高康养服务的覆盖率，还能提升居民的健康满意度。

养老机构：优化结果显示，一些区域如延庆区、密云区等需要增加养老机构的数量。这反映了这些区域的老龄人口比例较高，对养老设施的需求较大。增加养老机构可以更好地满足老年人的康养需求，提高养老服务的公平性和可达性。

公园绿地：模型建议在多个区域增加公园绿地数量，如海淀区、房山区等。公园绿地不仅有助于改善环境质量，还能为居民提供休闲和运动的空间，对提升居民的健康水平有积极作用。

文化设施：优化结果表明，文化设施在多个区域需要增加，如平谷区、延庆区等。文化设施的增加可以丰富居民的精神文化生活，提升居民的健康满意度。

(2) 实施策略建议

①政策建议

区域差异化政策：根据不同区域的康养资源现状和优化需求，制定差异化的政策。例如，对于医疗设施不足的区域，政府可以出台政策鼓励社会资本参与医疗设施建设；对于养老机构需求大的区域，可以提供土地和财政支持，吸引养老机构入驻。

跨区域合作：鼓励资源丰富的区域与资源不足的区域开展合作，实现资源共享和优势互补。例如，可以通过建立医联体、养老联盟等方式，提高资源利用效率。

政策引导：政府可以通过税收优惠、补贴等政策手段，引导企业和社会力量参与康养设施建设，推动康养产业的发展。

②资金投入方向

重点区域投入：根据模型结果，优先将资金投入到优化需求较大的区域，如房山区、平谷区、延庆区等。重点支持医疗设施、养老机构、公园绿地和文化设施的建设。

多元资金来源：除了政府财政投入外，积极引入社会资本、慈善基金等多元资金来源。通过 PPP 模式、政府引导基金等方式，拓宽资金渠道，确保康养设施建设的资金需求。

长期规划：制定长期的资金投入计划，确保康养设施建设的持续性和稳定性。根据区域发展需求和居民康养需求的变化，适时调整资金投入方向和规模。

③技术创新措施

智慧康养：利用大数据、人工智能、物联网等技术，建设智慧康养平台。通过平台实现医疗资源的优化配置、养老服务的精准推送、公园绿地的智能管理等，提高康养服务的效率和质量。

远程医疗：在医疗资源不足的区域，推广远程医疗服务。通过远程诊断、远程治疗等方式，缓解医疗资源分布不均衡的问题，提高医疗服务的可达性。

智能养老设施：引入智能养老设施，如智能健康监测设备、智能护理机器人等。这些设施可以提高养老服务的智能化水平，减轻护理人员的工作负担，提升老年人的生活质量。

5. 模型评价与推广

5.1 模型的优点

（1）贴合实际：本研究充分结合实际数据，考虑了康养资源分布的现状和居民健康状况的差异，通过熵值法和多目标优化算法，构建了合理的模型。模型结果能够为北京市康养城市建设提供科学依据。

（2）方法科学：模型运用了熵值法、TOPSIS 法、粒子群优化（PSO）、差分进化算法（DE）和非支配排序遗传算法（NSGAI II）等多种方法，将复杂的资源优化问题转化为可操作的数学模型，合理设置参数，确保模型的科学性和有效性。

（3）算法高效：使用的 PSO、DE 和 NSGAI II 算法具有全局搜索能力强、收敛速度快、计算效率高等优点，能够有效解决多目标优化问题，适用于康养资源优化配置的复杂场景。

（4）结果实用：模型得到的康养资源优化方案具有效率高、输出稳定、资源分配均衡等特点，基本不存在资源浪费、服务不均衡等问题，在现有条件下能有效提高康养服务的公平性和可达性。

5.2 模型的不足

（1）因素局限：实际应用中，政策变化、人口流动、经济发展等动态因素可能对康养资源的合理配置产生重要影响，但本文未能充分考虑这些动态因素的影响，一定程度上影响了模型的准确性和适应性。

（2）适用范围：本文提出的模型对于北京市的康养资源优化配置效果较好，但由于未对其他城市或地区进行验证，模型的适用范围可能较窄。对于其他城市或地区，可能需要根据当地实际情况进行调整。

（3）非线性因素：实际上，康养资源与居民健康之间的关系可能并非完全线性，还可能受到多种非线性因素的影响，而本文将其作为线性因子处理，忽略了边际效应的影响。

5.3 模型的推广

在其他城市或地区应用时，可以将模型中的参数（如人口数量、资源总量、服务半径等）替换成当地的实际情况，从而解决当地的康养资源优化配置问题；结合参考文献中关于动态系统建模的方法，进一步考虑政策变化、人口流动等因素的影响，引入动态规划或时间序列分析，使模型能够更好地适应动态变化的环境；技术拓展。结合大数据和人工智能技术，进一步优化模型的求解算法，提高模型的计算效率和精度。例如，可以引入深度学习算法对康养资源的需求进行预测，为资源优化配置提供更准确的输入数据。

参考文献

- [1] 刘建兵.北京市西城区探索大城市智慧康养服务[J].中国信息界,2020,(01):72-75.
- [2] 刘秀萍,李新宇,赵松婷,等.北京市广阳谷城市森林公园康养环境监测评价[J].北京农学院学报,2022,37(03):79-87.
- [3] 钟志刚,李丽娜,刘潇霞,等.基于 TOPSIS 法和 RSR 法模糊联合法的四川省县级公立医院上消化道癌机会性筛查质量评价[J].肿瘤预防与治疗,2025,38(02):132-137.
- [4] 周鹏飞,张思颖,苗艳凤.基于 AHP-TOPSIS 法的办公空间折叠床设计研究[J].林产工业,2025,62(03):63-70.
- [5] 李艺春,刘泽娇,洪艺天,等.基于多智能体强化学习的博弈综述[J].自动化学报,2025,51(03):540-558.
- [6] 徐钰龙,李君,李正权,等.基于深度强化学习的多用户蜂窝网络能效优化[J].计算机工程与设计,2025,46(03):734-740.
- [7] 王浩宇,李清,许泽华,等.基于 NSGA-III 的 eVTOL 飞行器双三相电机分层优化设计[J/OL]. 航空工程进展, 2025:1-9.
- [8] 吕焱豪,王桂荣,吴俊明.基于 NSGA-III 算法的六轴工业机器人逆运动学研究[J].组合机床与自动化加工技术,2025,(05):78-81+88.
- [9] 陈振,冉庆杰,英晓洋,等.基于 NSGA- II 和 TOPSIS 法的横波可控震源振动器平板疲劳寿命优化[J].工程设计学报,2025,32(02):272-280.
- [10] 叶恒舟,甘国军,董明刚.基于混合变量差分进化算法的多目标电动车辆路径优化[J/OL].桂林理工大学学报,2025:1-10.

附录

附录 1

第一问求解代码

首先安装并导入 pandas、geopandas、matplotlib、os、contextlib 等相关 python 库，然后下载北京曲线 shp 文件，关键代码如下：

①绘制北京市相关指标密度图

```
def read_excel_data(file_path, sheet_name=0):
    """读取 Excel 数据，提取需要的列"""
    try:
        df = pd.read_excel(file_path, sheet_name=sheet_name)
        # 确保数据包含必要的列
        required_columns = ['区', '医疗设施数量（家）', '养老机构数量（家）',
                             '公园绿地数量（个）', '文化设施数量（家）']
        missing_columns = [col for col in required_columns if col not in
                             df.columns]
        if missing_columns:
            raise ValueError(f"Excel 文件缺少必要的列: {'',
                              ''.join(missing_columns)}")

        # 提取需要的列
        data = df[required_columns].copy()
    def merge_data(shapefile_data, excel_data, district_column='NAME'):
        """合并地理数据和 Excel 数据"""
        try:
            # 确保区域名称匹配
            # 创建区域名称映射字典
            name_mapping = {
                # 如果 shapefile 和 Excel 中的区域名称不一致，在这里添加映射关系
                '东城区': '东城区',
                '西城区': '西城区',
                '朝阳区': '朝阳区',
                '丰台区': '丰台区',
                '石景山区': '石景山区',
                '海淀区': '海淀区',
                '门头沟区': '门头沟区',
                '房山区': '房山区',
                '通州区': '通州区',
                '顺义区': '顺义区',
                '昌平区': '昌平区',
                '大兴区': '大兴区',
                '怀柔区': '怀柔区',
                '平谷区': '平谷区',
```

```

        '密云县': '密云区',
        '延庆县': '延庆区',
    }

    # 应用名称映射
    if name_mapping:
        shapefile_data[district_column] =
shapefile_data[district_column].map(
            lambda x: name_mapping.get(x, x)
        )

    # 合并数据
    merged_data = shapefile_data.merge(
        excel_data,
        left_on=district_column,
        right_on='区',
        how='left'
    )

    # 检查合并结果
    missing_districts = merged_data[merged_data['医疗设施数量（家）
'].isna()][district_column].tolist()
    if missing_districts:
        print(f"警告：以下 {len(missing_districts)} 个区域在 Excel 数据中找
不到匹配：{'、'.join(missing_districts)}")
        print("这些区域将显示为无数据")

    return merged_data
except Exception as e:
    print(f"合并数据时出错：{e}")
    return None
def plot_four_panel_map(data, output_path=None, figsize=(20, 18)):
    """绘制四宫格分布图"""
    try:
        # 定义要绘制的列及其对应的标题和颜色映射
        columns_to_plot = {
            '医疗设施数量（家）': {'title': '北京市各区医疗设施数量分布', 'cmap':
'Reds'},
            '养老机构数量（家）': {'title': '北京市各区养老机构数量分布', 'cmap':
'Purples'},
            '公园绿地数量（个）': {'title': '北京市各区公园绿地数量分布', 'cmap':
'Greens'},
            '文化设施数量（家）': {'title': '北京市各区文化设施数量分布', 'cmap':
'Blues'}
        }
    
```

```

# 创建 2x2 的子图
fig, axes = plt.subplots(2, 2, figsize=figsize, sharex=True,
sharey=True)
axes = axes.flatten() # 将二维数组展平为一维

# 绘制每个子图
for i, (column, info) in enumerate(columns_to_plot.items()):
    ax = axes[i]

    # 准备绘图数据，处理可能的缺失值
    plot_data = data.copy()
    plot_data[column] = plot_data[column].fillna(0)

    # 创建颜色映射
    cmap = plt.cm.get_cmap(info['cmap'])

    # 绘制地图
    plot_data.plot(column=column, cmap=cmap, linewidth=0.8, ax=ax,
                    edgecolor='0.8', legend=True,
                    legend_kwds={'label': f'{column}', 'orientation':
'horizontal',
                                'shrink': 0.8, 'pad': 0.02})

```

②SLSQP 优化代码

读取文件

```
excel_file = pd.ExcelFile(r'D:\2025 中青杯数学建模\A 题：康养城市建设\数据\问题
1 第四问.xlsx')
```

获取对应工作表中数据

```
df = excel_file.parse('Sheet1')
```

康养资源指数

```
index = df['康养资源指数'].values
```

平均可达时间

```
time = df['医疗设施平均可达时间（分钟）'].values
```

总康养资源指数

```
total_index = np.sum(index)
```

权重 a 和 b

```
a = 0.5
```

```
b = 1 - a
```

定义目标函数

```
def objective(x):
```



```

    variance = np.var(x)
    avg_time = np.mean(time)
    return a * variance + b * avg_time

# 定义约束条件
def constraint(x):
    return total_index - np.sum(x)

con = {'type': 'eq', 'fun': constraint}

# 初始猜测值
x0 = index.copy()

# 进行优化, 使用 SLSQP 方法
solution = minimize(objective, x0, constraints=con, method='SLSQP')

# 优化后的康养资源指数
optimized_index = solution.x

# 优化后的方差
optimized_variance = np.var(optimized_index)

# 损失函数值
loss_value = objective(optimized_index)

```

附录 2

第二问求解代码

①熵权加 TOPSIS 代码

```

class HealthCityEvaluationSystem:
    def __init__(self, data_path):
        """初始化评价系统"""
        self.data_path = data_path
        self.data = None
        self.processed_data = None
        self.weights = None
        self.evaluation_result = None

```

```

def load_data(self):
    """加载数据"""
    try:
        # 读取 Excel 文件
        excel_file = pd.ExcelFile(self.data_path)
        # 获取指定工作表中的数据
        self.data = excel_file.parse('Sheet1')
        print("数据加载成功")
        return self.data
    except Exception as e:
        print(f"数据加载失败: {e}")
        return None

def preprocess_data(self):
    """数据预处理"""
    if self.data is None:
        print("请先加载数据")
        return None

    # 复制原始数据
    df = self.data.copy()

    # 处理缺失值
    for col in df.select_dtypes(include=[np.number]).columns:
        if df[col].isnull().sum() > 0:
            # 使用中位数填充数值型缺失值
            df[col] = df[col].fillna(df[col].median())

    # 保存处理后的数据
    self.processed_data = df
    print("数据预处理完成")
    return self.processed_data

def construct_indicator_system(self):
    """构建指标体系"""
    if self.processed_data is None:
        print("请先预处理数据")

```

```
return None
```

```
# 定义指标体系（根据实际数据调整）
```

```
indicator_system = {
```

```
    '康养资源': [
```

```
        '医疗设施分布密度(家 / 平方千米)', '养老机构分布密度(家  
/ 平方千米)',
```

```
        '公园绿地分布密度(个 / 平方千米)', '文化设施分布密度(家  
/ 平方千米)',
```

```
        '每千人床位数'
```

```
    ],
```

```
    '居民健康状况': [
```

```
        '人均平均寿命（岁）', '慢性病发病率（%）',
```

```
        '健康满意度（10 分）', '慢性病过早死亡率（%）'
```

```
    ],
```

```
    '环境质量': [
```

```
        'PM2.5 年均浓度（ $\mu\text{g}/\text{m}^3$ ）'
```

```
    ],
```

```
    '经济发展水平': [
```

```
        '地区生产总值（亿元）', '人均可支配收入（元）',
```

```
        '2023 年常住人口（万人）'
```

```
    ]
```

```
}
```

```
# 定义指标类型（1 为正向指标，0 为负向指标）
```

```
indicator_types = {
```

```
    '医疗设施分布密度（家 / 平方千米）': 1,
```

```
    '养老机构分布密度（家 / 平方千米）': 1,
```

```
    '公园绿地分布密度（个 / 平方千米）': 1,
```

```
    '文化设施分布密度（家 / 平方千米）': 1,
```

```
    '每千人床位数': 1,
```

```
    '人均平均寿命（岁）': 1,
```

```
    '慢性病发病率（%）': 0,
```

```
    '健康满意度（10 分）': 1,
```

```
    '慢性病过早死亡率（%）': 0,
```

```
    'PM2.5 年均浓度（ $\mu\text{g}/\text{m}^3$ ）': 0,
```

```
    '地区生产总值（亿元）': 1,
```

```

        '人均可支配收入（元）': 1,
        '2023 年常住人口（万人）': 1
    }

    return indicator_system, indicator_types

def entropy_weight_method(self, data, indicator_types):
    """熵权法计算权重"""
    # 数据标准化
    normalized_data = data.copy()

    for col in data.columns:
        if indicator_types[col] == 1: # 正向指标
            normalized_data[col] = (data[col] - data[col].min()) /
(data[col].max() - data[col].min())
        else: # 负向指标
            normalized_data[col] = (data[col].max() - data[col]) /
(data[col].max() - data[col].min())

    # 避免对数计算时出现零值
    normalized_data = normalized_data.replace(0, 1e-10)

    # 计算概率矩阵
    p = normalized_data / normalized_data.sum()

    # 计算熵值
    e = -1 / np.log(len(p)) * np.sum(p * np.log(p), axis=0)

    # 计算权重，处理可能的除零错误
    sum_weights = np.sum(1 - e)
    if sum_weights == 0:
        print("警告：所有指标的熵值为 1，无法计算权重，将使用等权重")
    weights = pd.Series(1 / len(e), index=e.index)
    else:
        weights = (1 - e) / sum_weights

```

```

        return weights

    def evaluate(self):
        """综合评价"""
        if self.processed_data is None:
            print("请先预处理数据")
            return None

        # 获取指标体系和指标类型
        indicator_system, indicator_types = self.construct_indicator_system()

        # 提取评价数据
        valid_columns = [col for col in indicator_types.keys() if col in
self.processed_data.columns]
        if len(valid_columns) != len(indicator_types):
            missing = set(indicator_types.keys()) -
set(self.processed_data.columns)
            print(f'警告：数据中缺少以下指标: {missing}')

        evaluation_data = self.processed_data[valid_columns]

        # 计算权重
        weights = self.entropy_weight_method(evaluation_data, indicator_types)
        self.weights = weights

        # 数据标准化
        normalized_data = evaluation_data.copy()
        for col in evaluation_data.columns:
            if indicator_types[col] == 1: # 正向指标
                normalized_data[col] = (evaluation_data[col] -
evaluation_data[col].min()) / (
                    evaluation_data[col].max() - evaluation_data[col].min())
            else: # 负向指标
                normalized_data[col] = (evaluation_data[col].max() -
evaluation_data[col]) / (
                    evaluation_data[col].max() - evaluation_data[col].min())

```

```

# 计算综合得分
scores = np.dot(normalized_data, weights)

# 计算各维度得分
dimension_scores = {}
for dimension, indicators in indicator_system.items():
    valid_dim_indicators = [ind for ind in indicators if ind in
valid_columns]

    if not valid_dim_indicators:
        print(f"警告：维度 '{dimension}' 中没有有效的指标")
        dimension_scores[dimension] = 0
        continue

    dimension_weights = weights[valid_dim_indicators]
    dimension_data = normalized_data[valid_dim_indicators]
    dimension_scores[dimension] = np.dot(dimension_data,
dimension_weights / dimension_weights.sum())

# 整理结果
result = self.processed_data[['区名']].copy()
result['综合得分'] = scores
for dimension, score in dimension_scores.items():
    result[f'{dimension}得分'] = score

# 排序
result = result.sort_values(by='综合得分', ascending=False)
result['排名'] = range(1, len(result) + 1)

self.evaluation_result = result
return result

def analyze_results(self):
    """结果分析"""
    if self.evaluation_result is None:
        print("请先进行评价")
        return None

```

```

result = self.evaluation_result

# 输出所有城市的排名
print("康养城市发展水平排名： ")
print(result)

# 可视化综合得分分布
plt.figure(figsize=(10, 6))
sns.histplot(result['综合得分'], kde=True)
plt.title('康养城市综合得分分布')
plt.xlabel('综合得分')
plt.ylabel('频数')
plt.savefig('综合得分分布.png')
plt.close()

# 可视化各维度得分关系
dimensions = [col for col in result.columns if col.endswith('得分') and
col != '综合得分']
plt.figure(figsize=(12, 10))
sns.pairplot(result[dimensions], diag_kind='kde')
plt.suptitle('各维度得分关系', y=1.02)
plt.savefig('各维度得分关系.png')
plt.close()

# 相关性分析
corr_matrix = result[dimensions + ['综合得分']].corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('各维度与综合得分相关性分析')
plt.savefig('相关性分析.png')
plt.close()

# 聚类分析
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
result['聚类类别'] = kmeans.fit_predict(result[dimensions])

```

```

# 可视化聚类结果
plt.figure(figsize=(10, 6))
sns.scatterplot(x=result['康养资源得分'], y=result['居民健康状况得分'],
                hue=result['聚类类别'], palette='viridis')
plt.title('康养城市聚类分析')
plt.savefig('聚类分析.png')
plt.close()

# 输出聚类结果统计
cluster_stats = result.groupby('聚类类别').agg({
    '综合得分': ['mean', 'min', 'max'],
    '康养资源得分': 'mean',
    '居民健康状况得分': 'mean',
    '环境质量得分': 'mean',
    '经济发展水平得分': 'mean',
    '区名': 'count'
})
print("\n 聚类结果统计： ")
print(cluster_stats)

return {
    'all_cities': result,
    'cluster_stats': cluster_stats
}

def export_results(self, output_path='康养城市评价结果.xlsx'):
    """导出结果"""
    if self.evaluation_result is None:
        print("请先进行评价")
        return None

    with pd.ExcelWriter(output_path) as writer:
        self.evaluation_result.to_excel(writer, sheet_name='综合评价结果',
index=False)

        if self.weights is not None:
            weights_df = pd.DataFrame({

```



```

        '指标': self.weights.index,
        '权重': self.weights.values
    })
    weights_df.to_excel(writer, sheet_name=' 指 标 权 重 ',
index=False)

    print(f'评价结果已导出至 {output_path}')
    return output_path

```

②强化学习代码

```

class PolicyNetwork(nn.Module):
    """策略网络，用于生成动作概率分布"""

    def __init__(self, state_dim, action_dim, hidden_dim=128):
        super(PolicyNetwork, self).__init__()
        self.fc1 = nn.Linear(state_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.mean = nn.Linear(hidden_dim, action_dim)
        self.log_std = nn.Linear(hidden_dim, action_dim)

        # 初始化层参数
        for m in self.modules():
            if isinstance(m, nn.Linear):
                nn.init.orthogonal_(m.weight, gain=np.sqrt(2))
                nn.init.constant_(m.bias, 0.0)

        # 限制标准差范围
        self.log_std_min = -20
        self.log_std_max = 2

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))

        # 计算动作均值
        mean = self.mean(x)

        # 计算动作标准差(限制范围)

```

```

        log_std = self.log_std(x)
        log_std = torch.clamp(log_std, self.log_std_min, self.log_std_max)
        std = torch.exp(log_std)

        return mean, std

def get_action(self, state):
    state = torch.FloatTensor(state)
    mean, std = self.forward(state)

    # 从正态分布采样动作
    dist = Normal(mean, std)
    action = dist.sample()

    # 计算动作对数概率
    log_prob = dist.log_prob(action).sum(-1)

    return action.detach().numpy(), log_prob.detach()

def evaluate(self, state, action):
    mean, std = self.forward(state)
    dist = Normal(mean, std)

    # 计算动作对数概率和熵
    log_prob = dist.log_prob(action).sum(-1, keepdim=True)
    entropy = dist.entropy().sum(-1, keepdim=True)

    return log_prob, entropy

class ValueNetwork(nn.Module):
    """价值网络，用于估计状态价值"""

    def __init__(self, state_dim, hidden_dim=128):
        super(ValueNetwork, self).__init__()
        self.fc1 = nn.Linear(state_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)

```

```

        self.value = nn.Linear(hidden_dim, 1)

    # 初始化层参数
    for m in self.modules():
        if isinstance(m, nn.Linear):
            nn.init.orthogonal_(m.weight, gain=np.sqrt(2))
            nn.init.constant_(m.bias, 0.0)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        value = self.value(x)
        return value

class PPO:
    """PPO 算法实现"""

    def __init__(self, state_dim, action_dim, hidden_dim=128, lr=3e-4,
gamma=0.99,
                    eps_clip=0.2, k_epochs=10, entropy_coef=0.01):
        self.gamma = gamma
        self.eps_clip = eps_clip
        self.k_epochs = k_epochs
        self.entropy_coef = entropy_coef

    # 初始化策略网络和价值网络
    self.policy = PolicyNetwork(state_dim, action_dim, hidden_dim)
    self.value = ValueNetwork(state_dim, hidden_dim)

    # 优化器
    self.policy_optimizer = optim.Adam(self.policy.parameters(), lr=lr)
    self.value_optimizer = optim.Adam(self.value.parameters(), lr=lr)

    # 存储轨迹数据
    self.states = []
    self.actions = []

```

```

        self.log_probs = []
        self.rewards = []
        self.is_terminals = []

    def select_action(self, state):
        action, log_prob = self.policy.get_action(state)
        self.states.append(state)
        self.actions.append(action)
        self.log_probs.append(log_prob)
        return action

    def store_transition(self, reward, is_terminal):
        self.rewards.append(reward)
        self.is_terminals.append(is_terminal)

    def update(self):
        # 计算折扣回报
        rewards = []
        discounted_return = 0
        for reward, is_terminal in zip(reversed(self.rewards),
reversed(self.is_terminals)):
            if is_terminal:
                discounted_return = 0
            discounted_return = reward + (self.gamma * discounted_return)
            rewards.insert(0, discounted_return)

        # 标准化回报
        rewards = torch.FloatTensor(rewards).unsqueeze(1)
        rewards = (rewards - rewards.mean()) / (rewards.std() + 1e-7)

        # 转换数据为张量
        states = torch.FloatTensor(np.array(self.states))
        actions = torch.FloatTensor(np.array(self.actions))
        old_log_probs = torch.FloatTensor(np.array(self.log_probs)).unsqueeze(1)

        # 优化策略和价值网络
        for _ in range(self.k_epochs):

```

```

# 获取当前策略下的动作对数概率和熵
log_probs, entropy = self.policy.evaluate(states, actions)

# 计算策略比率
ratio = torch.exp(log_probs - old_log_probs)

# 计算优势估计
state_values = self.value(states)
advantages = rewards - state_values.detach()

# 计算策略损失 (PPO-Clip)
surr1 = ratio * advantages
surr2 = torch.clamp(ratio, 1 - self.eps_clip, 1 + self.eps_clip) *
advantages

policy_loss = -torch.min(surr1, surr2).mean() - self.entropy_coef *
entropy.mean()

# 计算价值损失
value_loss = ((state_values - rewards) ** 2).mean()

# 优化策略网络
self.policy_optimizer.zero_grad()
policy_loss.backward()
self.policy_optimizer.step()

# 优化价值网络
self.value_optimizer.zero_grad()
value_loss.backward()
self.value_optimizer.step()

# 清空轨迹缓冲区
self.states = []
self.actions = []
self.log_probs = []
self.rewards = []
self.is_terminals = []
return policy_loss.item(), value_loss.item()

```

分

```
class HealthCityEnv:
    """康养城市环境模拟"""

    def __init__(self, data):
        # 使用标准化后的数据
        self.data = data
        self.reset()

        # 定义状态空间和动作空间维度
        self.state_dim = self.data.shape[1]
        # 动作对应各指标的调整比例
        self.action_dim = self.data.shape[1]

        # 定义康养指标权重 (可根据专家意见调整)
        self.weights = {
            '医疗设施密度': 0.1,
            '养老机构密度': 0.1,
            '公园绿地密度': 0.1,
            '慢性病发病率': -0.15, # 负权重表示降低该指标会提高康养得

            '健康满意度': 0.15,
            '文化设施密度': 0.08,
            '慢性病过早死亡率': -0.12, # 负权重
            '地区生产总值': 0.07,
            '人均可支配收入': 0.1,
            'PM2.5 年均浓度': -0.1, # 负权重
            '每千人床位数': 0.1
        }

        # 确保权重总和为 1
        weight_sum = sum([abs(w) for w in self.weights.values()])
        for key in self.weights:
            self.weights[key] /= weight_sum

    def reset(self):
```

```

        # 随机选择一个区作为初始状态
        self.current_idx = np.random.randint(0, len(self.data))
        self.current_state = self.data.iloc[self.current_idx].values
        self.year = 2023 # 模拟从 2023 年开始
        self.done = False
        return self.current_state.copy()

    def step(self, action):
        # 应用动作 (调整各指标)
        # 动作范围在[-1, 1]之间, 表示增减比例
        action = np.clip(action, -0.5, 0.5) # 限制调整范围为±50%
        new_state = self.current_state * (1 + action)

        # 计算康养得分作为奖励
        reward = self._calculate_health_score(new_state)

        # 更新年份
        self.year += 1

        # 判断是否结束 (模拟 5 年的规划周期)
        if self.year >= 2028:
            self.done = True

        # 更新当前状态
        self.current_state = new_state

        return new_state.copy(), reward, self.done, {}

    def _calculate_health_score(self, state):
        """计算康养综合得分"""
        score = 0
        for i, col in enumerate(self.data.columns):
            if col in self.weights:
                score += state[i] * self.weights[col]
        return score

```

```

# 读取 Excel 数据
try:
    # 尝试读取用户的数据文件
    df = pd.read_excel(r"C:\Users\Admin\Desktop\数据 2.xlsx")
    print("成功读取数据文件")
    print(f"数据包含 {df.shape[0]} 行 {df.shape[1]} 列")
    print(f"列名: {list(df.columns)}")
except Exception as e:
    print(f"读取数据时出错: {e}")
    # 创建模拟数据用于演示
    print("使用模拟数据进行演示...")
    df = pd.DataFrame({
        '区名': [f'区{i+1}' for i in range(16) for _ in range(5)],
        '年份': [2018, 2019, 2020, 2021, 2022] * 16,
        '医疗设施密度': np.random.uniform(0.1, 20, 80),
        '养老机构密度': np.random.uniform(0.01, 0.6, 80),
        '公园绿地密度': np.random.uniform(0.02, 2.15, 80),
        '慢性病发病率': np.random.randint(24, 31, 80),
        '健康满意度': np.random.uniform(7.2, 9.2, 80),
        '文化设施密度': np.random.uniform(0.02, 5.98, 80),
        '慢性病过早死亡率': np.random.uniform(9.8, 12.0, 80),
        '地区生产总值': np.random.uniform(230, 10206, 80),
        '人均可支配收入': np.random.randint(58000, 93000, 80),
        'PM2.5 年均浓度': np.random.randint(30, 40, 80),
        '每千人床位数': np.random.uniform(2.4, 13.5, 80),
        '2023 年常住人口': np.random.uniform(33, 344, 80)
    })

# 数据预处理
# 提取相关指标列
relevant_columns = [
    '医疗设施密度', '养老机构密度', '公园绿地密度',
    '慢性病发病率', '健康满意度', '文化设施密度',
    '慢性病过早死亡率', '地区生产总值', '人均可支配收入',
    'PM2.5 年均浓度', '每千人床位数'
]

```



```

# 提取 2022 年的数据作为基础状态
try:
    data_2022 = df[df['年份'] == 2022][relevant_columns].copy()
    print(f'成功提取 2022 年数据，包含 {data_2022.shape[0]} 行')
except:
    # 如果没有 2022 年数据，使用最新年份
    latest_year = df['年份'].max()
    data_2022 = df[df['年份'] == latest_year][relevant_columns].copy()
    print(f'使用最新年份({latest_year})的数据，包含 {data_2022.shape[0]} 行')

# 数据标准化
for col in data_2022.columns:
    if col in ['慢性病发病率', '慢性病过早死亡率', 'PM2.5 年均浓度']:
        # 这些指标值越低越好，采用反向归一化
        data_2022[col] = (data_2022[col].max() - data_2022[col]) /\
            (data_2022[col].max() - data_2022[col].min() +
1e-8)
    else:
        # 其他指标值越高越好，采用正向归一化
        data_2022[col] = (data_2022[col] - data_2022[col].min()) /\
            (data_2022[col].max() - data_2022[col].min() +
1e-8)

# 创建环境
env = HealthCityEnv(data_2022)

# 初始化 PPO 代理
agent = PPO(
    state_dim=env.state_dim,
    action_dim=env.action_dim,
    hidden_dim=128,
    lr=3e-4,
    gamma=0.99,
    eps_clip=0.2,
    k_epochs=10,
    entropy_coef=0.01

```

```

)

# 训练参数
max_episodes = 500
max_timesteps = 5 # 每个 episode 5 年
print_interval = 10
save_interval = 100

# 记录训练过程
episode_rewards = []
policy_losses = []
value_losses = []

# 训练循环
for episode in range(max_episodes):
    state = env.reset()
    episode_reward = 0

    for t in range(max_timesteps):
        # 选择动作
        action = agent.select_action(state)

        # 执行动作
        next_state, reward, done, _ = env.step(action)

        # 存储转换
        agent.store_transition(reward, done)

        # 更新状态
        state = next_state
        episode_reward += reward

    if done:
        break

# 更新策略
policy_loss, value_loss = agent.update()

```

```

# 记录训练过程
episode_rewards.append(episode_reward)
policy_losses.append(policy_loss)
value_losses.append(value_loss)

# 打印训练信息
if (episode + 1) % print_interval == 0:
    avg_reward = np.mean(episode_rewards[-print_interval:])
    print(f"Episode {episode + 1}/{max_episodes} - Avg Reward: {avg_reward:.4f}")

# 保存模型
if (episode + 1) % save_interval == 0:
    torch.save(agent.policy.state_dict(), f'ppo_policy_{episode + 1}.pth')
    torch.save(agent.value.state_dict(), f'ppo_value_{episode + 1}.pth')

```

附录 3

第三问求解代码

Pso 算法

读取问题 1 的数据

```

def load_data(file_path):
    try:
        data = pd.read_excel(file_path)
        print(f"成功加载数据，包含{len(data)}个区域")
        return data
    except Exception as e:
        print(f"加载数据出错：{e}")
        return None

```

定义康养设施选址问题

```

class HealthcareFacilityProblem(Problem):
    def __init__(self, data, n_facilities=16, n_max_facilities=8):
        # 数据预处理
        self.data = data

```

```

self.n_regions = len(data)
self.n_facilities = n_facilities # 候选设施点数量
self.n_max_facilities = n_max_facilities # 最多可建设的设施数量

# 提取关键数据
self.coordinates = data[['经度', '纬度']].values
self.demand = data['慢性病发病率(%)'].values
self.population = data['2023 年常住人口(万人)'].values
self.gdp = data['地区生产总值(亿元)'].values
self.existing_medical = data['医疗设施数量(家)'].values
self.existing_elderly = data['养老机构数量(家)'].values
self.existing_parks = data['公园绿地数量(个)'].values
self.existing_culture = data['文化设施数量(家)'].values
self.pm25 = data['PM2.5 年均浓度( $\mu\text{g}/\text{m}^3$ )'].values

# 计算距离矩阵
self.distance_matrix = np.zeros((self.n_regions, self.n_regions))
for i in range(self.n_regions):
    for j in range(self.n_regions):
        self.distance_matrix[i, j] = np.sqrt(
            (self.coordinates[i, 0] - self.coordinates[j, 0]) ** 2 +
            (self.coordinates[i, 1] - self.coordinates[j, 1]) ** 2
        )

# 标准化数据
self.scaler = MinMaxScaler()
self.existing_medical_norm =
self.scaler.fit_transform(self.existing_medical.reshape(-1, 1)).flatten()
self.existing_elderly_norm =
self.scaler.fit_transform(self.existing_elderly.reshape(-1, 1)).flatten()
self.existing_parks_norm =
self.scaler.fit_transform(self.existing_parks.reshape(-1, 1)).flatten()
self.existing_culture_norm =
self.scaler.fit_transform(self.existing_culture.reshape(-1, 1)).flatten()

super().__init__(
    n_var=self.n_regions,
    n_obj=4, # 四个目标函数
    n_constr=2, # 两个约束条件
    xl=0, # 决策变量下界
    xu=1, # 决策变量上界
    vtype=int # 决策变量类型为整数
)

def _evaluate(self, x, out, *args, **kwargs):
    # 初始化目标函数值和约束条件值

```

```

f1 = np.zeros(len(x)) # 服务覆盖率（最大化）
f2 = np.zeros(len(x)) # 资源分配均衡度（最小化）
f3 = np.zeros(len(x)) # 总成本（最小化）
f4 = np.zeros(len(x)) # 环境适宜性（最大化）
g1 = np.zeros(len(x)) # 约束 1: 至少建设 5 个设施
g2 = np.zeros(len(x)) # 约束 2: 预算约束

# 预算参数
max_budget = np.sum(self.gdp) * 0.4 # 预算上限为 GDP 总和的 40%

for i in range(len(x)):
    # 获取当前解选择的设施
    selected_facilities = np.where(x[i] == 1)[0]

    # 目标 1: 计算服务覆盖率
    if len(selected_facilities) > 0:
        # 计算每个区域到最近设施的距离
        min_distances = np.min(self.distance_matrix[:,
selected_facilities], axis=1)
        # 假设服务半径为 0.1（约 11 公里）
        covered = (min_distances <= 0.1).astype(int)
        # 加权覆盖率（基于慢性病发病率）
        f1[i] = -np.sum(covered * self.demand) /
np.sum(self.demand) # 取负号，因为要最大化
    else:
        f1[i] = 0

    # 目标 2: 计算资源分配均衡度（使用 Gini 系数）
    if len(selected_facilities) > 0:
        # 计算现有设施分布的 Gini 系数
        total_existing = (self.existing_medical_norm +
self.existing_elderly_norm +
self.existing_parks_norm +
self.existing_culture_norm)

        # 计算选中设施的资源分布
        resource_distribution = np.zeros(self.n_regions)
        resource_distribution[selected_facilities] = 1
        # 计算 Gini 系数
        f2[i] = self.calculate_gini(resource_distribution)
    else:
        f2[i] = 1 # 最大不平等

    # 目标 3: 计算总成本（与 GDP 相关）
    f3[i] = np.sum(self.gdp[selected_facilities]) * 0.01 # 假设建设
成本为 GDP 的 1%

```

```

        # 目标 4: 计算环境适宜性
        if len(selected_facilities) > 0:
            # PM2.5 浓度越低越好, 取负号
            f4[i] = np.sum(self.pm25[selected_facilities]) /
len(selected_facilities)
        else:
            f4[i] = 0

        # 约束 1: 至少建设 5 个设施
        g1[i] = 5 - len(selected_facilities)

        # 约束 2: 预算约束
        g2[i] = f3[i] - max_budget

    # 设置目标函数和约束条件
    out["F"] = np.column_stack([f1, f2, f3, f4])
    out["G"] = np.column_stack([g1, g2])

def calculate_gini(self, array):
    """计算 Gini 系数, 衡量资源分配的不平等程度"""
    array = array.flatten()
    if np.sum(array) == 0:
        return 0
    array = array / np.sum(array)
    array = np.sort(array)
    index = np.arange(1, len(array) + 1)
    return np.sum((2 * index - len(array) - 1) * array) / (len(array) *
np.sum(array))

```

